Relatório Completo do Estudo 2

Marcos Belo Jefferson Amorim

8 de novembro de 2026

1 Estudo 2 — Análise de Sinais e Sistemas nos Domínios da Frequência e da Transformada

Instituição: Instituto Federal da Paraíba — IFPB
Disciplina: Processamento Digital de Sinais
Professor: Moacy Pereira da Silva
Equipe: Marcos Vinicius Belo da Silva e Jefferson Amorim da Costa

1.1 Introdução

Fourier, Laplace e Z conectam tempo, frequência e planos complexos. O estudo reúne os 18 tópicos do guia e uma aplicação de sensor industrial. Os resultados computacionais são reproduzíveis pelos notebooks incluídos. Os sinais dos projetos são sintéticos.

2 Frequência e Exponenciais Complexas

---

3 Resumo conceitual

Os sinais oscilatórios podem ser representados por funções senoidais e exponenciais complexas, que são ferramentas fundamentais no Processamento Digital de Sinais. Uma senoide discreta é caracterizada por três parâmetros principais: amplitude, frequência angular e fase. A amplitude determina o valor máximo da oscilação, a frequência angular indica a rapidez com que o sinal varia ao longo das amostras e a fase representa o deslocamento inicial da senoide. A relação entre senoides e exponenciais complexas é estabelecida pela relação de Euler, que permite expressar funções trigonométricas utilizando números complexos. Essa representação facilita operações matemáticas e a análise de sinais, especialmente no estudo das transformadas de Fourier.
Uma exponencial complexa pode ser interpretada como um sinal cuja parte real corresponde a um cosseno e cuja parte imaginária corresponde a um seno. Dessa forma, as duas componentes podem ser analisadas separadamente para compreender o comportamento do sinal no plano complexo.

---

4 Formulação matemática

4.1 Representação de uma senoide discreta

Uma senoide discreta pode ser representada por:

$$x[n] = A\cos(\omega_0 n + \phi) \tag{1}$$

Em que:

- $x[n]$: valor do sinal na amostra n;
- A: amplitude do sinal;
- ω_0: frequência angular em radianos por amostra;
- n: índice da amostra;
- ϕ: fase inicial, em radianos.

A amplitude determina a intensidade da oscilação. A frequência angular controla a variação do sinal entre as amostras, enquanto a fase modifica a posição inicial da senoide no eixo horizontal.
A frequência angular está relacionada à frequência em ciclos por amostra pela expressão:

$$\omega_0 = 2\pi f \tag{2}$$

Quando a frequência é expressa em hertz e existe uma frequência de amostragem f_s, a relação é:

$$\omega_0 = 2\pi \frac{f}{f_s} \tag{3}$$

4.2 Relação de Euler

A relação de Euler estabelece uma conexão entre as funções trigonométricas e as exponenciais complexas:

$$e^{j\theta} = \cos(\theta) + j\sin(\theta) \tag{4}$$

Em que:

- e: número de Euler;
- j: unidade imaginária, com $j^2 = -1$;
- θ: ângulo em radianos;
- $\cos(\theta)$: parte real;
- $\sin(\theta)$: parte imaginária.

Aplicando essa relação a uma frequência angular discreta:

$$e^{j\omega_0 n} = \cos(\omega_0 n) + j\sin(\omega_0 n) \tag{5}$$

Assim, uma exponencial complexa reúne duas componentes oscilatórias: uma real e outra imaginária.

4.3 Representação de cosseno e seno por exponenciais complexas

A partir da relação de Euler, podemos obter as expressões:

$$\cos(\omega_0 n) = \frac{1}{2}\left(e^{j\omega_0 n} + e^{-j\omega_0 n}\right) \tag{6}$$

E:

$$\sin(\omega_0 n) = \frac{1}{2j}\left(e^{j\omega_0 n} - e^{-j\omega_0 n}\right) \tag{7}$$

Portanto, uma senoide pode ser representada pela combinação de duas exponenciais complexas com frequências angulares opostas.

4.4 Representação complexa de uma senoide com amplitude e fase

Considerando uma senoide com amplitude A e fase inicial ϕ, sua representação complexa pode ser escrita como:

$$z[n] = Ae^{j(\omega_0 n+\phi)} \tag{8}$$

Aplicando a relação de Euler:

$$z[n] = A\cos(\omega_0 n + \phi) + jA\sin(\omega_0 n + \phi) \tag{9}$$

Consequentemente:

$$\text{Re}\{z[n]\} = A\cos(\omega_0 n + \phi) \tag{10}$$

$$\text{Im}\{z[n]\} = A\sin(\omega_0 n + \phi) \tag{11}$$

A parte real corresponde à senoide cossenoidal, enquanto a parte imaginária corresponde à senoide senoidal.

---

5 Exemplo

Considere o seguinte sinal discreto:

$$x[n] = 2\cos\left(\frac{\pi}{4}n + \frac{\pi}{6}\right) \tag{12}$$

Nesse exemplo, os parâmetros são:

- Amplitude: $A = 2$;
- Frequência angular: $\omega_0 = \frac{\pi}{4}$ rad/amostra;
- Fase inicial: $\phi = \frac{\pi}{6}$ rad.

Para a amostra $n = 0$, temos:

$$x[0] = 2\cos\left(\frac{\pi}{4}\cdot 0 + \frac{\pi}{6}\right) \tag{13}$$

$$x[0] = 2\cos\left(\frac{\pi}{6}\right) \tag{14}$$

$$x[0] = \sqrt{3} \approx 1{,}732 \tag{15}$$

A representação complexa correspondente é:

$$z[n] = 2e^{j\left(\frac{\pi}{4}n+\frac{\pi}{6}\right)} \tag{16}$$

Sua parte real representa o sinal original, enquanto a parte imaginária representa a componente senoidal associada.
Esse exemplo demonstra que a exponencial complexa permite representar simultaneamente duas componentes oscilatórias, mantendo as informações de amplitude, frequência e fase.

---

6 Simulação

6.1 Objetivo da simulação

A simulação tem como objetivo analisar o comportamento de senoides discretas e exponenciais complexas, verificando como a amplitude, a frequência e a fase influenciam a representação dos sinais.
Serão realizadas quatro análises:

1. Comparação de senoides com diferentes frequências;
2. Comparação de sinais com diferentes fases;
3. Comparação de sinais com diferentes amplitudes;
4. Representação das partes real e imaginária de uma exponencial complexa.

6.2 Bibliotecas utilizadas

Para desenvolver a simulação, serão utilizadas as bibliotecas NumPy, Matplotlib e Seaborn.

- **NumPy:** criação das sequências e realização dos cálculos matemáticos;
- **Matplotlib:** construção dos gráficos;
- **Seaborn:** configuração visual dos gráficos.

6.3 Implementação

Apresentar o código-fonte utilizado na simulação.
Código da simulação:

```
# ============================================================
# IMPORTAÇÕES E CONFIGURAÇÕES
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(
    style="whitegrid",
    context="notebook"
)

# ============================================================
# PARÂMETROS GERAIS
# ============================================================

n = np.arange(0, 41, 1)

# Frequências angulares em radianos por amostra
frequencias = [
    np.pi / 8,
    np.pi / 4,
    np.pi / 2
]

# Amplitude e fase iniciais
A = 1
phi = 0

# ============================================================
# 1. SENOides COM DIFERENTES FREQUÊNCIAS
# ============================================================

plt.figure(figsize=(12, 8))

for i, omega in enumerate(frequencias, 1):
```

```
    x = A * np.cos(omega * n + phi)

    plt.subplot(3, 1, i)

    plt.stem(n, x)

    plt.title(
        rf"Senoide discreta: $\omega_0={omega/np.pi:.2f}\pi$"
    )

    plt.xlabel("Índice da amostra (n)")
    plt.ylabel("Amplitude")

    plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 2. ALTERAÇÃO DA FASE
# ============================================================

omega = np.pi / 4

fases = [
    0,
    np.pi / 4,
    np.pi / 2
]

plt.figure(figsize=(12, 5))

for fase in fases:

    x = A * np.cos(omega * n + fase)

    plt.plot(
        n,
        x,
        marker="o",
        label=rf"$\phi={fase/np.pi:.2f}\pi$"
    )

plt.title("Influência da fase na senoide discreta")
plt.xlabel("Índice da amostra (n)")
plt.ylabel("Amplitude")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ============================================================
# 3. ALTERAÇÃO DA AMPLITUDE
# ============================================================

amplitudes = [
    0.5,
    1.0,
    2.0
]

plt.figure(figsize=(12, 5))

for amplitude in amplitudes:

    x = amplitude * np.cos(omega * n + phi)

    plt.plot(
        n,
        x,
        marker="o",
        label=f"A = {amplitude}"
    )
```

```
plt.title("Influência da amplitude na senoide discreta")
plt.xlabel("Índice da amostra (n)")
plt.ylabel("Amplitude")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ============================================================
# 4. EXPONENCIAL COMPLEXA
# ============================================================

omega = np.pi / 4
A = 1
phi = 0

z = A * np.exp(1j * (omega * n + phi))

parte_real = np.real(z)
parte_imaginaria = np.imag(z)

plt.figure(figsize=(12, 8))

# Parte real
plt.subplot(2, 1, 1)

plt.stem(
    n,
    parte_real,
    linefmt="C0 -",
    markerfmt="C0o",
    basefmt="k -"
)

plt.title("Parte real da exponencial complexa")
plt.xlabel("Índice da amostra (n)")
plt.ylabel("Parte real")

plt.grid(True)

# Parte imaginária
plt.subplot(2, 1, 2)

plt.stem(
    n,
    parte_imaginaria,
    linefmt="C1 -",
    markerfmt="C1o",
    basefmt="k -"
)

plt.title("Parte imaginária da exponencial complexa")
plt.xlabel("Índice da amostra (n)")
plt.ylabel("Parte imaginária")

plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 5. REPRESENTAÇÃO NO PLANO COMPLEXO
# ============================================================

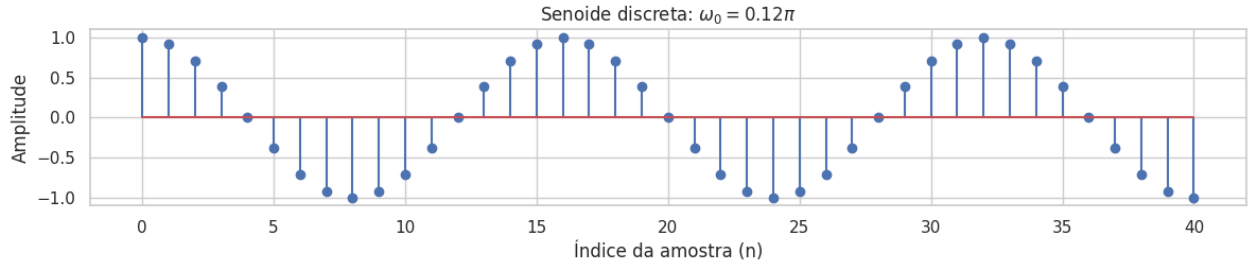
plt.figure(figsize=(7, 7))

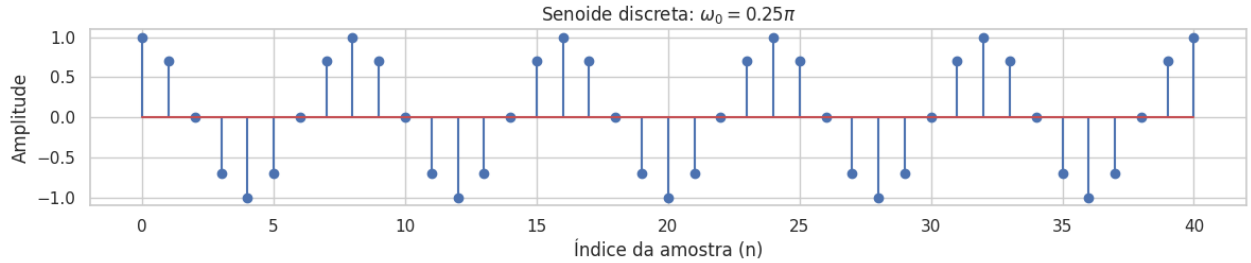
plt.plot(
    parte_real,
    parte_imaginaria,
    marker="o"
)

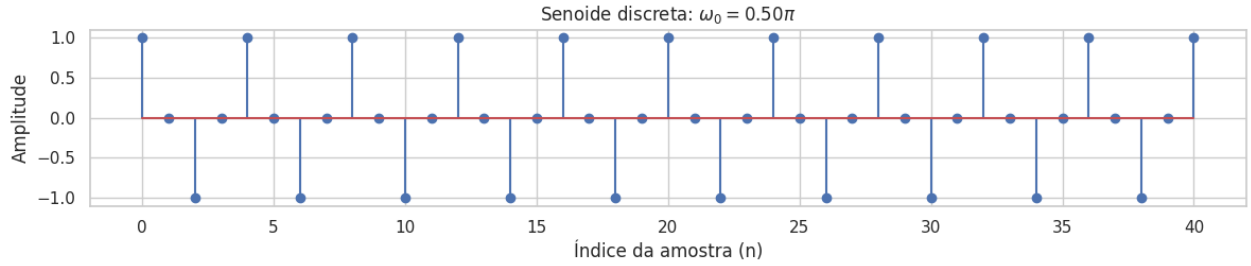
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(0, color="black", linewidth=0.8)
```

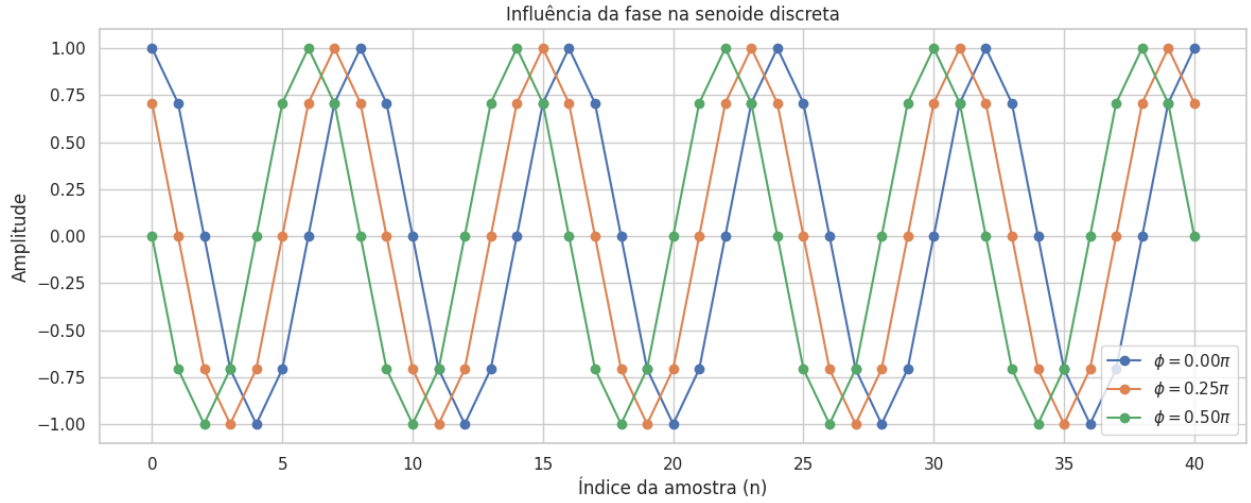
```
plt.title("Exponencial complexa no plano complexo")
plt.xlabel("Parte real")
plt.ylabel("Parte imaginária")

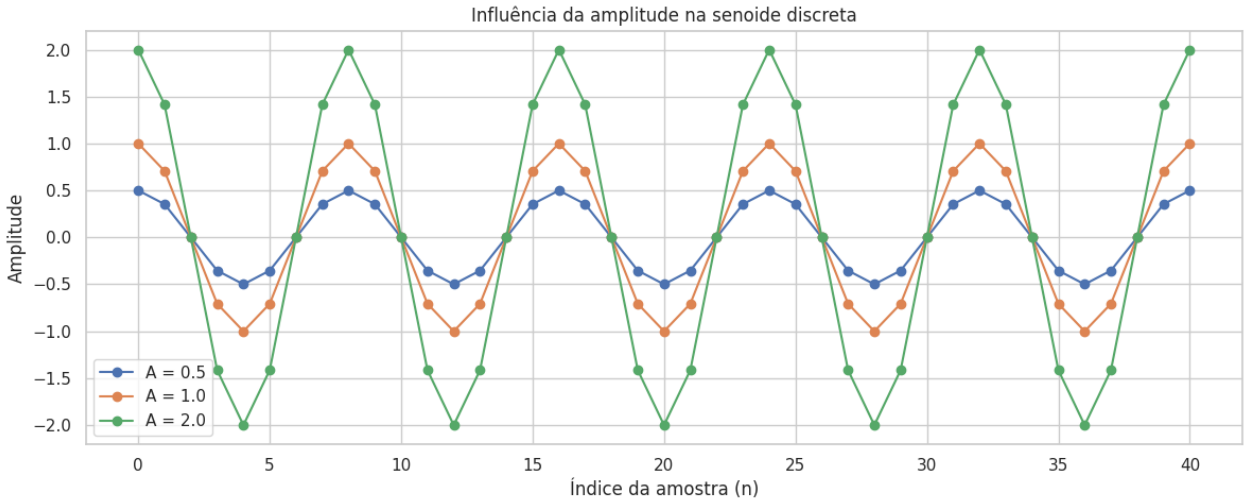
plt.axis("equal")
plt.grid(True)
plt.tight_layout()
plt.show()
```

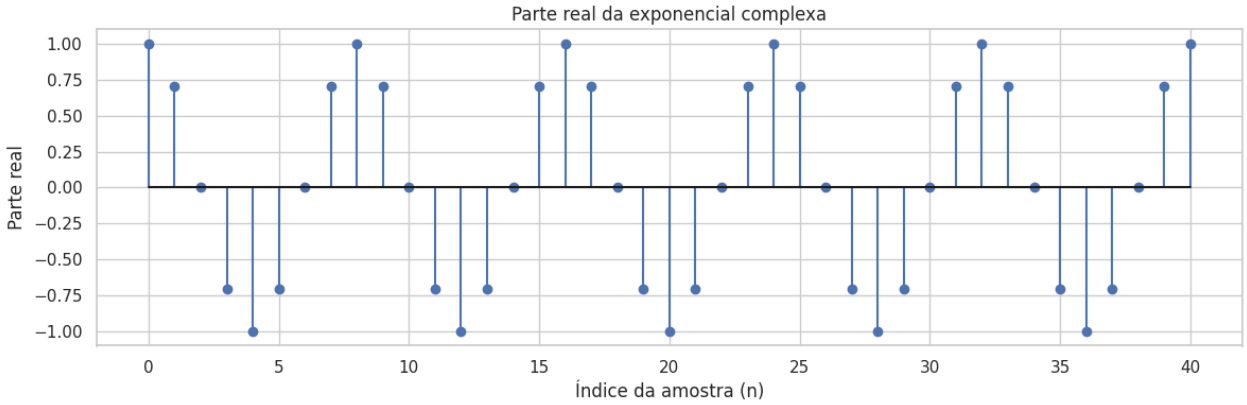


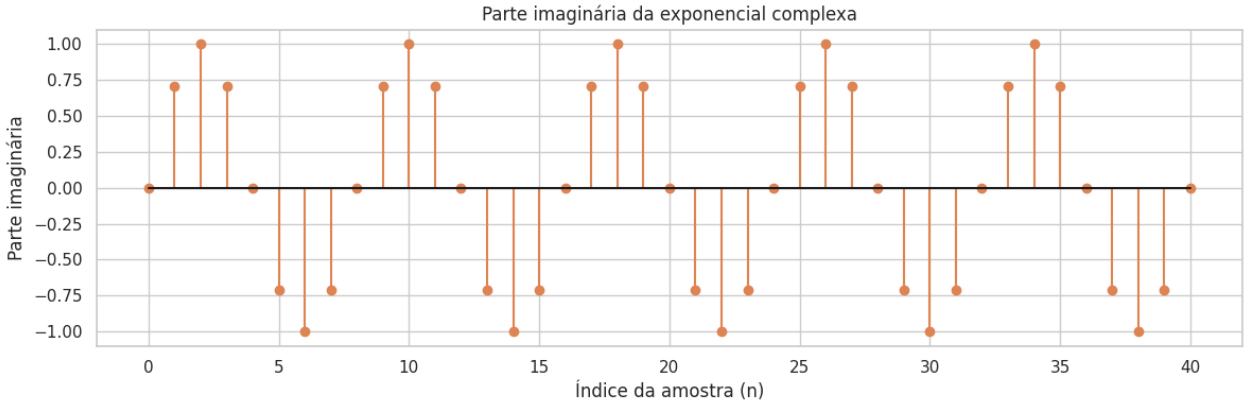


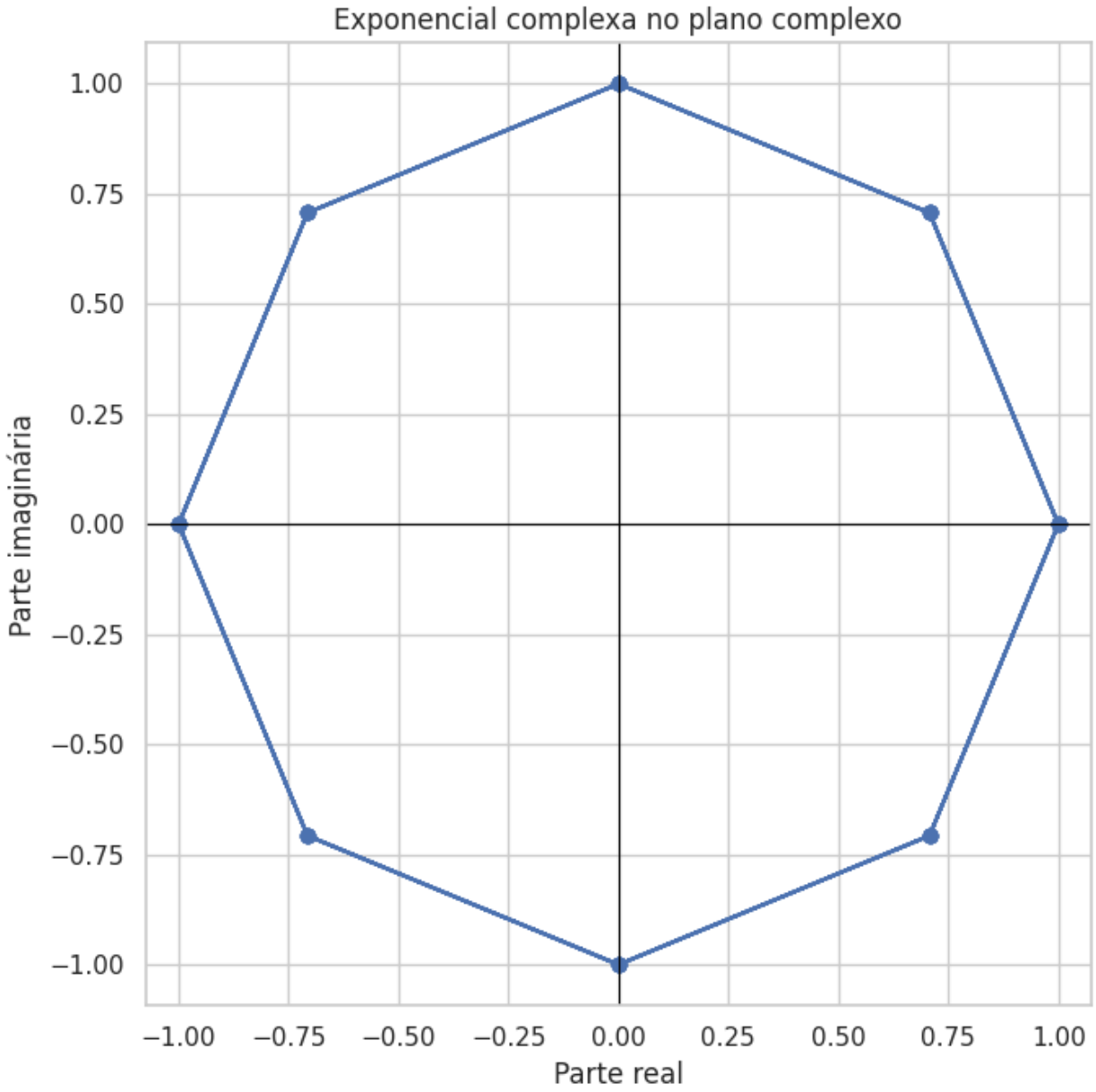





Influência da amplitude na senoide discreta
Amplitude
Índice da amostra (n)
A = 0.5
A = 1.0
A = 2.0


Parte real da exponencial complexa
Parte real
Índice da amostra (n)


Parte imaginária da exponencial complexa
Parte imaginária
Índice da amostra (n)



---

7 Resultados gráficos

A execução da simulação produz cinco conjuntos de representações gráficas, permitindo observar diferentes características dos sinais.

7.1 Senoides com diferentes frequências

O primeiro conjunto apresenta três senoides com frequências angulares de $\frac{\pi}{8}$, $\frac{\pi}{4}$ e $\frac{\pi}{2}$ radianos por amostra.
Espera-se observar que o aumento da frequência angular provoca mais oscilações em uma mesma quantidade de amostras. A senoide de menor frequência apresenta variações mais lentas, enquanto a de maior frequência apresenta oscilações mais rápidas.

7.2 Influência da fase

O segundo gráfico apresenta senoides com a mesma amplitude e frequência, mas com fases iniciais diferentes: 0, $\frac{\pi}{4}$ e $\frac{\pi}{2}$.
As curvas mantêm a mesma amplitude e o mesmo padrão de oscilação, porém apresentam deslocamentos entre si. Isso demonstra que a fase modifica a posição inicial do sinal sem alterar sua frequência ou amplitude.

7.3 Influência da amplitude

O terceiro gráfico compara sinais com amplitudes de $0{,}5$, $1{,}0$ e $2{,}0$.
Observa-se que o aumento da amplitude amplia os valores máximos e mínimos da senoide. Apesar dessa alteração, a frequência e a fase permanecem constantes.

7.4 Partes real e imaginária

O quarto conjunto apresenta separadamente a parte real e a parte imaginária da exponencial complexa.
A parte real possui comportamento de cosseno, enquanto a parte imaginária possui comportamento de seno. Ambas apresentam a mesma frequência angular, mas possuem uma diferença de fase de $\frac{\pi}{2}$.

7.5 Representação no plano complexo

O último gráfico apresenta os valores da exponencial complexa no plano formado pelos eixos real e imaginário.
Como a amplitude utilizada é igual a 1, os pontos pertencem à circunferência unitária. Conforme o índice das amostras aumenta, os pontos percorrem essa circunferência, representando a evolução da fase da exponencial complexa.

---

8 Interpretação

Os resultados demonstram que amplitude, frequência e fase são parâmetros independentes que determinam características específicas de uma senoide discreta.
A amplitude controla a intensidade da oscilação, enquanto a frequência angular determina a rapidez com que ocorrem as variações entre as amostras. A fase, por sua vez, estabelece o deslocamento inicial do sinal, sem modificar sua amplitude ou frequência.
A representação por exponenciais complexas permite reunir as componentes cossenoidal e senoidal em uma única expressão matemática. A parte real reproduz o comportamento do cosseno, enquanto a parte imaginária representa o seno.
A visualização no plano complexo também evidencia que uma exponencial complexa de amplitude constante percorre uma circunferência de raio igual à amplitude. O ângulo associado a cada amostra corresponde à fase instantânea do sinal.
Dessa maneira, a simulação confirma as relações matemáticas apresentadas e demonstra como a representação complexa facilita a análise de sinais oscilatórios.

---

9 Discussão crítica

A representação de sinais por exponenciais complexas é uma ferramenta importante no Processamento Digital de Sinais, pois simplifica operações matemáticas e estabelece a base para técnicas de análise no domínio da frequência, como as Transformadas de Fourier.

A utilização das partes real e imaginária permite compreender que uma exponencial complexa não representa apenas uma operação matemática abstrata, mas uma forma compacta de descrever duas componentes oscilatórias relacionadas.
Entretanto, é importante considerar que, em sinais discretos, frequências angulares separadas por múltiplos de 2π produzem as mesmas sequências exponenciais. Além disso, uma senoide discreta somente será periódica quando a razão entre sua frequência angular e 2π for racional.
Outro aspecto importante é que a análise realizada considera sinais ideais, sem a presença de ruídos ou outras interferências. Em aplicações reais, esses fatores podem modificar os sinais observados e exigir técnicas adicionais de processamento.
Portanto, o estudo das frequências e exponenciais complexas fornece uma base matemática essencial para compreender as próximas etapas da análise espectral, especialmente o desenvolvimento das Transformadas de Fourier, que utilizam exponenciais complexas para representar e analisar sinais em diferentes frequências.

10 Transformada de Fourier em Tempo Discreto (DTFT)

---

11 Resumo conceitual

A Transformada de Fourier em Tempo Discreto (DTFT) é uma ferramenta matemática utilizada para analisar o conteúdo em frequência de sinais discretos. Enquanto a representação no domínio do tempo mostra como os valores de uma sequência variam em função do índice das amostras, a DTFT permite observar quais frequências estão presentes no sinal e como elas contribuem para sua composição.
A DTFT transforma uma sequência discreta $x[n]$ em uma representação no domínio da frequência angular, identificada por $X(e^{j\omega})$. Essa representação é, em geral, uma função complexa e periódica em relação à frequência angular, com período de 2π.
A análise espectral normalmente utiliza duas grandezas: o espectro de magnitude e o espectro de fase. A magnitude indica a intensidade das componentes presentes em cada frequência, enquanto a fase representa o deslocamento angular dessas componentes.
Dessa forma, a DTFT permite compreender características de um sinal que nem sempre são facilmente identificadas apenas pela observação de sua sequência no domínio do tempo, sendo fundamental para o estudo de filtragem, análise espectral e processamento digital de sinais.

---

12 Formulação matemática

12.1 Definição da DTFT

A Transformada de Fourier em Tempo Discreto de uma sequência $x[n]$ é definida por:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{+\infty} x[n]e^{-j\omega n} \tag{17}$$

Em que:

- $X(e^{j\omega})$: representação do sinal no domínio da frequência;
- $x[n]$: sequência discreta original;
- n: índice da amostra;
- ω: frequência angular, em radianos por amostra;
- j: unidade imaginária;

- $e^{-j\omega n}$: exponencial complexa utilizada na transformação.

A expressão realiza uma combinação das amostras do sinal com exponenciais complexas de diferentes frequências. O resultado permite identificar a contribuição de cada frequência para a composição do sinal.

12.2 Transformada inversa

A recuperação da sequência original a partir de sua representação espectral é realizada pela Transformada Inversa de Fourier em Tempo Discreto:

$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} \, d\omega \tag{18}$$

Essa expressão demonstra que é possível reconstruir o sinal discreto a partir de seu espectro, considerando um período completo da representação em frequência.

12.3 Espectro de magnitude

Como a DTFT apresenta valores complexos, sua magnitude é calculada pelo módulo da representação:

$$|X(e^{j\omega})| = \sqrt{\text{Re}\{X(e^{j\omega})\}^2 + \text{Im}\{X(e^{j\omega})\}^2} \tag{19}$$

A magnitude representa a intensidade das componentes espectrais em cada frequência angular.

12.4 Espectro de fase

A fase é determinada pelo argumento do número complexo:

$$\angle X(e^{j\omega}) = \text{atan2}\left(\text{Im}\{X(e^{j\omega})\}, \text{Re}\{X(e^{j\omega})\}\right) \tag{20}$$

A função atan2 permite determinar o ângulo considerando os sinais das partes real e imaginária, identificando corretamente o quadrante.

12.5 Periodicidade da DTFT

A DTFT apresenta periodicidade de 2π radianos por amostra:

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}) \tag{21}$$

Isso significa que o espectro se repete a cada intervalo de 2π, sendo comum representar a DTFT no intervalo de $-\pi$ a π.

---

13 Exemplo

Para compreender a aplicação da DTFT, considere a seguinte sequência discreta:

$$x[n] = \begin{cases} 1, & 0 \leq n \leq 3 \\ 0, & \text{caso contrário} \end{cases} \tag{22}$$

A sequência possui quatro amostras iguais a 1:

$$x[n] = [1, 1, 1, 1] \tag{23}$$

Aplicando a definição da DTFT:

$$X(e^{j\omega}) = \sum_{n=0}^{3} e^{-j\omega n} \tag{24}$$

Expandindo o somatório:

$$X(e^{j\omega}) = 1 + e^{-j\omega} + e^{-j2\omega} + e^{-j3\omega} \tag{25}$$

Essa expressão pode ser simplificada utilizando a soma de uma progressão geométrica:

$$X(e^{j\omega}) = \frac{1 - e^{-j4\omega}}{1 - e^{-j\omega}} \tag{26}$$

Para os pontos em que o denominador não é nulo, essa expressão pode ser reescrita como:

$$X(e^{j\omega}) = e^{-j\frac{3\omega}{2}} \frac{\sin(2\omega)}{\sin(\omega/2)} \tag{27}$$

Em frequências nas quais o denominador se anula, a expressão original deve ser utilizada por continuidade.
A magnitude é:

$$|X(e^{j\omega})| = \left| \frac{\sin(2\omega)}{\sin(\omega/2)} \right| \tag{28}$$

A fase, quando a magnitude é diferente de zero, é determinada pelo argumento da expressão complexa.
No caso de $\omega = 0$, todas as exponenciais são iguais a 1. Portanto:

$$X(e^{j0}) = 1 + 1 + 1 + 1 = 4 \tag{29}$$

Isso significa que a magnitude do espectro na frequência zero é igual a 4, indicando uma componente contínua associada à soma das amostras da sequência.
Os zeros do espectro ocorrem em:

$$\omega = \frac{k\pi}{2} \tag{30}$$

para k inteiro, exceto nos pontos equivalentes a múltiplos de 2π, nos quais a expressão apresenta o valor 4.
Esse exemplo demonstra como uma sequência simples no domínio do tempo pode ser representada por uma função complexa no domínio da frequência.

---

14 Simulação

14.1 Objetivo da simulação

A simulação tem como objetivo calcular numericamente a DTFT de uma sequência discreta e analisar seu comportamento no domínio da frequência.
Será utilizada uma sequência com quatro amostras unitárias, permitindo comparar sua representação temporal com os espectros de magnitude e fase.
As atividades desenvolvidas serão:

1. Criar e representar a sequência discreta no domínio do tempo;
2. Calcular numericamente sua DTFT;
3. Determinar o espectro de magnitude;
4. Determinar o espectro de fase;
5. Identificar as frequências nas quais ocorrem os zeros espectrais;
6. Verificar a periodicidade da DTFT.

14.2 Bibliotecas utilizadas

Para desenvolver a simulação, serão utilizadas as seguintes bibliotecas:

- **NumPy:** criação das sequências e realização dos cálculos numéricos;
- **Matplotlib:** construção e organização dos gráficos;
- **Seaborn:** configuração visual das representações gráficas.

14.3 Código da simulação

A DTFT será calculada diretamente a partir de sua definição matemática, utilizando um conjunto de valores de frequência angular. Essa abordagem permite observar a transformação sem depender de uma implementação pronta da DFT ou da FFT.

```
# ============================================================
# IMPORTAÇÕES E CONFIGURAÇÕES
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(
    style="whitegrid",
    context="notebook"
)

# ============================================================
# DEFINIÇÃO DA SEQUÊNCIA DISCRETA
# ============================================================

n = np.arange(0, 4)

x = np.array([1, 1, 1, 1])

# ============================================================
# DEFINIÇÃO DO EIXO DE FREQUÊNCIA
# ============================================================

omega = np.linspace(
    -np.pi,
    np.pi,
    2000
)

# ============================================================
# CÁLCULO NUMÉRICO DA DTFT
# ============================================================

X = np.zeros(len(omega), dtype=complex)

for i, w in enumerate(omega):

    X[i] = np.sum(
        x * np.exp( -1j * w * n)
    )

# ============================================================
# CÁLCULO DA MAGNITUDE E DA FASE
# ============================================================

magnitude = np.abs(X)

fase = np.angle(X)

# Identificação dos pontos com magnitude próxima de zero
# Evita interpretar como significativa a fase de pontos nulos.

mascara = magnitude > 1e -8

fase_plot = np.where(
```

```
    mascara,
    fase,
    np.nan
)

# ============================================================
# 1. REPRESENTAÇÃO NO DOMÍNIO DO TEMPO
# ============================================================

plt.figure(figsize=(10, 4))

plt.stem(
    n,
    x
)

plt.title("Sequência discreta no domínio do tempo")
plt.xlabel("Índice da amostra (n)")
plt.ylabel("Amplitude")

plt.xticks(np.arange( -1, 5))
plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 2. ESPECTRO DE MAGNITUDE
# ============================================================

plt.figure(figsize=(10, 4))

plt.plot(
    omega / np.pi,
    magnitude
)

plt.title("Espectro de magnitude da DTFT")
plt.xlabel(r"Frequência angular ($\omega/\pi$)")
plt.ylabel(r"$|X(e^{j\omega})|$")

plt.xticks(
    np.arange( -1, 1.1, 0.25)
)

plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 3. ESPECTRO DE FASE
# ============================================================

plt.figure(figsize=(10, 4))

plt.plot(
    omega / np.pi,
    fase_plot
)

plt.title("Espectro de fase da DTFT")
plt.xlabel(r"Frequência angular ($\omega/\pi$)")
plt.ylabel("Fase (rad)")

plt.xticks(
    np.arange( -1, 1.1, 0.25)
)

plt.yticks(
    [ -np.pi, -np.pi/2, 0, np.pi/2, np.pi],
    [
        r"$ -\pi$",
        r"$ -\pi/2$",
        "0",
```

```
        r"$\pi/2$",
        r"$\pi$"
    ]
)

plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 4. PERIODICIDADE DA DTFT
# ============================================================

omega_ext = np.linspace(
    -3 * np.pi,
    3 * np.pi,
    3000
)

X_ext = np.zeros(
    len(omega_ext),
    dtype=complex
)

for i, w in enumerate(omega_ext):

    X_ext[i] = np.sum(
        x * np.exp( -1j * w * n)
    )

plt.figure(figsize=(12, 4))

plt.plot(
    omega_ext / np.pi,
    np.abs(X_ext)
)

plt.title("Periodicidade do espectro da DTFT")
plt.xlabel(r"Frequência angular ($\omega/\pi$)")
plt.ylabel(r"$|X(e^{j\omega})|$")

plt.xticks(
    np.arange( -3, 3.1, 0.5)
)

plt.grid(True)

plt.tight_layout()
plt.show()

# ============================================================
# 5. VALORES IMPORTANTES DO ESPECTRO
# ============================================================

print("ANÁLISE DA DTFT")
print("=" * 45)

print(f"Sequência: {x}")
print(f"Número de amostras: {len(x)}")

indice_maximo = np.argmax(magnitude)

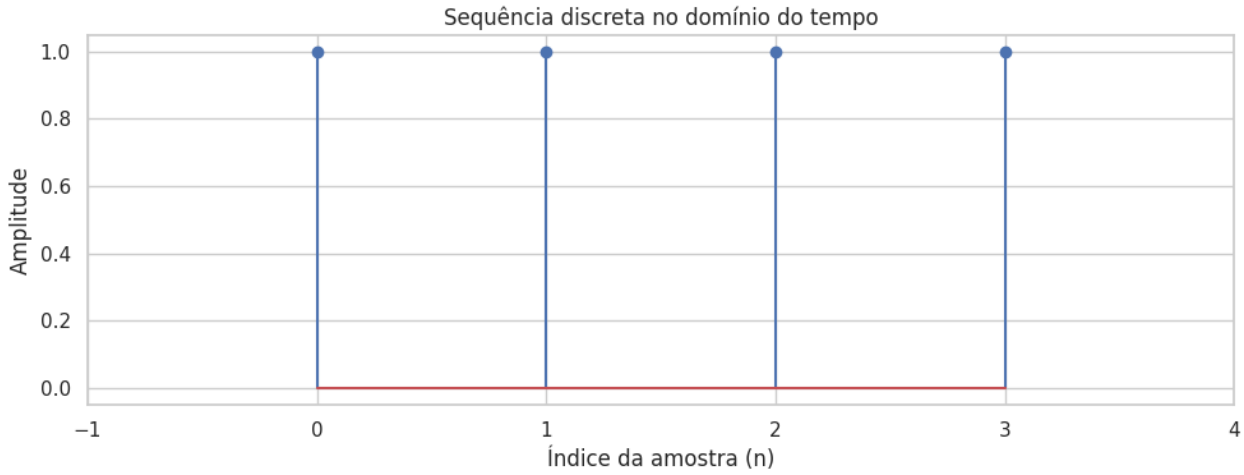
print(
    f"Maior magnitude: {magnitude[indice_maximo]:.4f}"
)

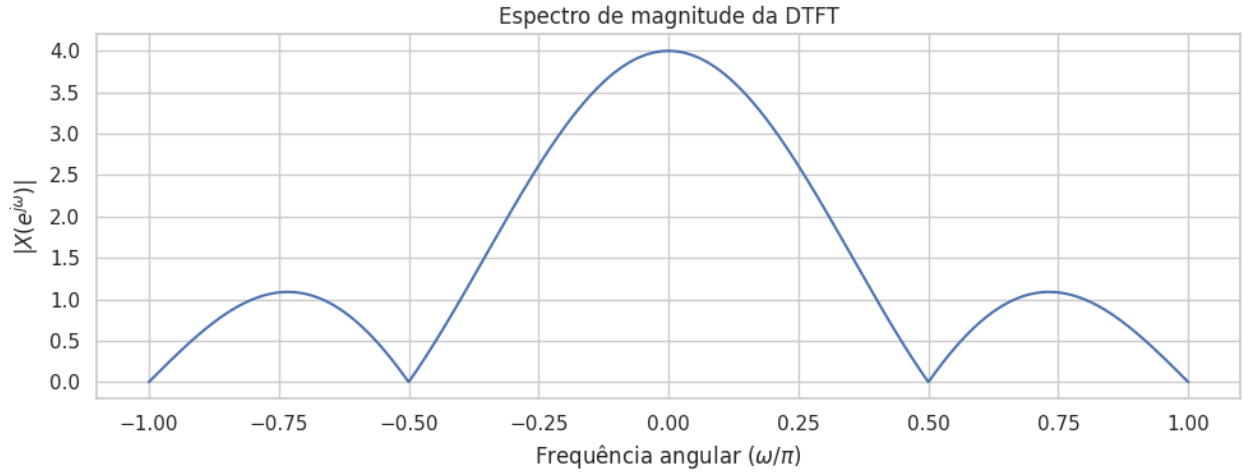
print(
    f"Frequência do máximo: "
    f"{omega[indice_maximo]/np.pi:.4f} * pi rad/amostra"
)

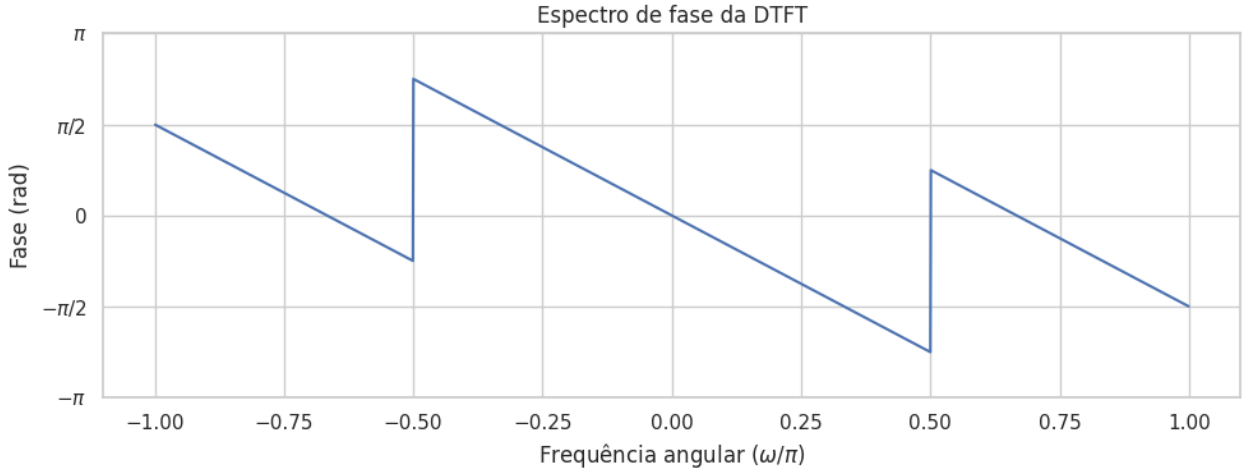
print(
    f"Valor da DTFT em omega = 0: "
    f"{np.sum(x):.4f}"
```

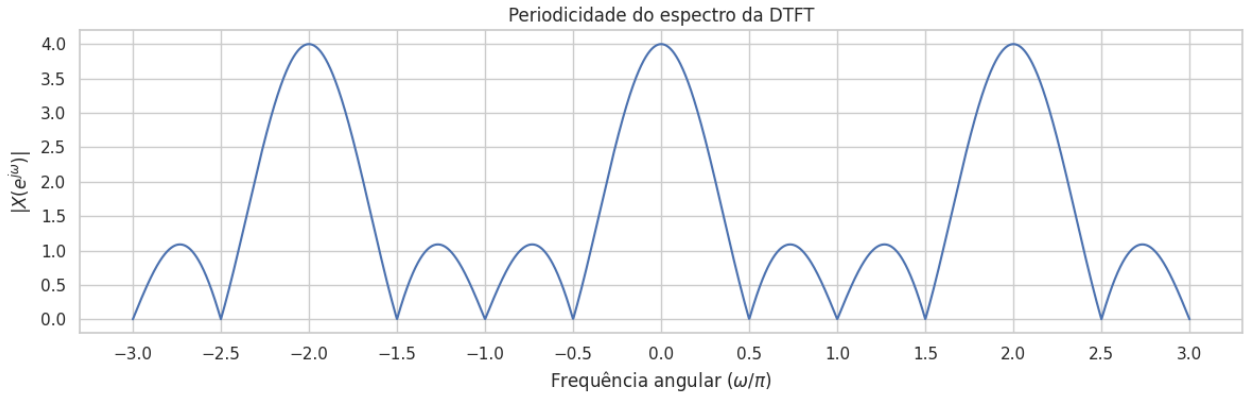
```
)

print("\nFrequências dos zeros teóricos:")
print(" - pi, - pi/2, pi/2, pi e seus múltiplos equivalentes,")
print("exceto nos múltiplos de 2*pi.")
```










```
ANÁLISE DA DTFT
==========================================
Sequência: [1 1 1 1]
Número de amostras: 4
Maior magnitude: 4.0000
Frequência do máximo: -0.0005 * pi rad/amostra
Valor da DTFT em omega = 0: 4.0000

Frequências dos zeros teóricos:
 - pi, - pi/2, pi/2, pi e seus múltiplos equivalentes,
exceto nos múltiplos de 2*pi.
```

---

15 Resultados gráficos

A execução da simulação produz cinco representações que permitem analisar o comportamento da sequência e de sua transformada.

15.1 Sequência no domínio do tempo

O primeiro gráfico apresenta as quatro amostras da sequência, todas com amplitude igual a 1.
A representação permite identificar diretamente a quantidade de amostras e seus respectivos valores. Entretanto, não apresenta de forma explícita a distribuição das componentes em frequência.

15.2 Espectro de magnitude

O segundo gráfico apresenta a magnitude da DTFT em função da frequência angular.
Espera-se observar o maior valor de magnitude na frequência zero, com amplitude igual a 4.
Também são identificados zeros espectrais em $-\pi/2$ e $\pi/2$, dentro do intervalo apresentado.
Esses pontos indicam frequências nas quais as contribuições das amostras se cancelam na soma das exponenciais complexas.

15.3 Espectro de fase

O terceiro gráfico apresenta a fase da DTFT.
A fase permite observar o comportamento angular das componentes complexas em cada frequência. Como a fase é indefinida quando a magnitude é zero, esses pontos são omitidos da representação para evitar interpretações incorretas.
Além disso, podem ocorrer mudanças de fase de aproximadamente π radianos quando o espectro real muda de sinal.

15.4 Periodicidade da DTFT

O quarto gráfico apresenta o espectro em um intervalo maior de frequência, permitindo observar sua repetição.
A periodicidade de 2π radianos por amostra é uma característica fundamental da DTFT. Portanto, os padrões observados entre $-\pi$ e π voltam a aparecer nos intervalos seguintes.

15.5 Valores numéricos

O último trecho do código apresenta informações numéricas sobre a sequência e sua transformada, incluindo a maior magnitude e o valor da DTFT na frequência zero.
Esses valores permitem verificar numericamente os resultados apresentados nos gráficos e comparar a simulação com a formulação matemática.

---

16 Interpretação

A comparação entre os domínios do tempo e da frequência demonstra que cada representação fornece informações diferentes sobre o mesmo sinal.
No domínio do tempo, é possível observar a quantidade de amostras, seus valores e a organização da sequência. No domínio da frequência, a DTFT permite identificar como essas amostras contribuem para a composição espectral do sinal.
Para a sequência escolhida, o valor máximo da magnitude ocorre na frequência zero, sendo igual a 4. Isso acontece porque todas as amostras possuem o mesmo valor e, nessa frequência, as exponenciais complexas da transformação são iguais a 1.
Os zeros espectrais em $-\pi/2$ e $\pi/2$ indicam frequências nas quais as contribuições das amostras se anulam. Esse comportamento evidencia a utilidade da análise espectral para identificar características que não são imediatamente perceptíveis na representação temporal.
A fase complementa a análise da magnitude, fornecendo informações sobre o comportamento angular das componentes complexas. Entretanto, nos pontos em que a magnitude é nula, a fase não possui significado definido.
Por fim, a representação em um intervalo maior confirma a periodicidade da DTFT, demonstrando que seu espectro se repete a cada 2π radianos por amostra.

---

17 Discussão crítica

A Transformada de Fourier em Tempo Discreto é uma ferramenta essencial para compreender a composição em frequência de sinais discretos. Sua principal vantagem está na possibilidade de analisar separadamente as componentes espectrais por meio da magnitude e da fase, facilitando a identificação de frequências predominantes e características importantes do sinal.
Entretanto, a DTFT apresenta algumas limitações práticas. Embora sua definição matemática considere uma soma infinita, a simulação computacional geralmente trabalha com sequências finitas ou com sequências que permitem o cálculo da transformação. Além disso, a representação numérica utiliza uma quantidade limitada de pontos de frequência, de modo que a resolução visual depende da densidade da grade utilizada.
Outro aspecto importante é que a DTFT não deve ser confundida com a DFT ou a FFT. A DTFT representa o espectro como uma função contínua da frequência angular, enquanto a DFT fornece um conjunto finito de amostras desse espectro. A FFT, por sua vez, é um algoritmo eficiente para calcular a DFT.
No exemplo desenvolvido, a sequência possui apenas quatro amostras, o que facilita a análise matemática e computacional. Entretanto, sinais reais podem apresentar maior duração, ruídos, variações de amplitude e componentes de frequência sobrepostas, exigindo métodos de análise mais elaborados.

Portanto, o estudo da DTFT estabelece uma base importante para compreender a análise espectral e os métodos computacionais de processamento de sinais. A capacidade de observar magnitude, fase e periodicidade no domínio da frequência será fundamental para os próximos tópicos, especialmente aqueles relacionados à DFT, FFT e resolução espectral.

18 Transformada Discreta de Fourier (DFT)

---

19 Resumo conceitual

A Transformada Discreta de Fourier (DFT, do inglês *Discrete Fourier Transform*) é uma ferramenta matemática utilizada no Processamento Digital de Sinais para analisar o conteúdo em frequência de sequências discretas e finitas. Sua principal função é transformar um sinal representado no domínio do tempo em uma representação no domínio da frequência, permitindo identificar quais componentes de frequência estão presentes e suas respectivas contribuições.
Na prática computacional, os sinais digitais são armazenados como sequências finitas de amostras. Dessa forma, a DFT permite analisar essas sequências por meio de um conjunto de coeficientes complexos que representam a contribuição de diferentes frequências para a composição do sinal.
A DFT transforma uma sequência contendo N amostras em N coeficientes espectrais. Cada coeficiente $X[k]$ está associado a uma frequência específica e contém informações sobre a magnitude e a fase dessa componente.
A magnitude indica a intensidade de cada componente de frequência, enquanto a fase representa seu deslocamento angular em relação à referência utilizada na representação do sinal.
A Transformada Discreta de Fourier é amplamente utilizada em aplicações como análise espectral, processamento de áudio, telecomunicações, processamento de imagens, identificação de frequências e análise de vibrações.
Além disso, a DFT possui uma versão inversa, denominada IDFT (*Inverse Discrete Fourier Transform*), que permite reconstruir a sequência original a partir dos coeficientes espectrais.

---

20 Formulação matemática

20.1 Transformada Discreta de Fourier

A Transformada Discreta de Fourier de uma sequência $x[n]$, contendo N amostras, é definida por:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \tag{31}$$

Para:

$$k = 0, 1, 2, \ldots, N-1 \tag{32}$$

Em que:

- $X[k]$: coeficiente complexo da DFT associado ao índice de frequência k;
- $x[n]$: sequência original no domínio do tempo;
- N: número total de amostras;
- n: índice das amostras no domínio do tempo;
- k: índice correspondente aos coeficientes espectrais;

- j: unidade imaginária, sendo $j^2 = -1$.

O termo exponencial complexo pode ser desenvolvido utilizando a relação de Euler:

$$e^{-j\theta} = \cos(\theta) - j\sin(\theta) \tag{33}$$

Assim, a DFT realiza a comparação entre o sinal original e diferentes componentes exponenciais complexas, determinando a contribuição de cada frequência presente na sequência.

20.2 Transformada Discreta de Fourier inversa

A reconstrução do sinal original é realizada por meio da Transformada Discreta de Fourier inversa:

$$x[n] = \frac{1}{N}\sum_{k=0}^{N-1} X[k]e^{j\frac{2\pi}{N}kn} \tag{34}$$

Essa expressão demonstra que é possível recuperar a sequência original a partir dos coeficientes calculados pela DFT.

20.3 Magnitude e fase

Como os coeficientes da DFT são números complexos, cada um pode ser representado por uma parte real e uma parte imaginária:

$$X[k] = \mathrm{Re}\{X[k]\} + j\,\mathrm{Im}\{X[k]\} \tag{35}$$

A magnitude do espectro é calculada por:

$$|X[k]| = \sqrt{\mathrm{Re}\{X[k]\}^2 + \mathrm{Im}\{X[k]\}^2} \tag{36}$$

Já a fase é determinada por:

$$\phi[k] = \mathrm{atan2}\left(\mathrm{Im}\{X[k]\}, \mathrm{Re}\{X[k]\}\right) \tag{37}$$

A magnitude permite identificar a intensidade das componentes de frequência, enquanto a fase fornece informações sobre o deslocamento angular de cada componente.

20.4 Relação entre os índices da DFT e as frequências

A frequência associada a cada índice k é determinada por:

$$f_k = \frac{kf_s}{N} \tag{38}$$

Em que:

- f_k: frequência correspondente ao índice k, em hertz;
- f_s: frequência de amostragem, em hertz;
- N: número de amostras.

A resolução em frequência da DFT é dada por:

$$\Delta f = \frac{f_s}{N} \tag{39}$$

Portanto, quanto maior o número de amostras para uma mesma frequência de amostragem, menor será o espaçamento entre os pontos do espectro, proporcionando maior resolução em frequência.
Para sinais reais, o espectro apresenta simetria conjugada. Por esse motivo, é comum utilizar apenas a primeira metade do espectro, correspondente às frequências positivas, para facilitar a interpretação dos resultados.

---

21 Exemplo

Considere um sinal discreto formado pela soma de duas senoides com frequências diferentes:

$$x[n] = A_1 \cos(\omega_1 n) + A_2 \cos(\omega_2 n) \tag{40}$$

Para o exemplo, serão utilizados os seguintes parâmetros:

- Amplitude da primeira senoide: $A_1 = 1{,}0$;
- Amplitude da segunda senoide: $A_2 = 0{,}5$;
- Primeira frequência: $f_1 = 10\,\text{Hz}$;
- Segunda frequência: $f_2 = 30\,\text{Hz}$;
- Frequência de amostragem: $f_s = 256\,\text{Hz}$;
- Número de amostras: $N = 256$.

Considerando que a frequência angular discreta é calculada por:

$$\omega = \frac{2\pi f}{f_s} \tag{41}$$

As frequências angulares das duas componentes serão:

$$\omega_1 = \frac{2\pi \cdot 10}{256} \tag{42}$$

$$\omega_2 = \frac{2\pi \cdot 30}{256} \tag{43}$$

Assim, o sinal utilizado será:

$$x[n] = \cos\left(\frac{20\pi}{256} n\right) + 0{,}5 \cos\left(\frac{60\pi}{256} n\right) \tag{44}$$

Aplicando a DFT a essa sequência, espera-se identificar duas componentes principais no espectro de magnitude, localizadas em 10 Hz e 30 Hz.
Como a primeira componente possui amplitude igual a $1{,}0$ e a segunda possui amplitude igual a $0{,}5$, espera-se que o primeiro pico apresente o dobro da magnitude do segundo no espectro unilateral corretamente normalizado.
Além disso, como as duas senoides foram definidas sem deslocamento de fase, suas fases iniciais são iguais a zero.

---

22 Simulação

A simulação será desenvolvida utilizando Python, com as bibliotecas NumPy, Matplotlib e Seaborn.
O objetivo é gerar um sinal composto por duas senoides, calcular sua DFT, obter os espectros de magnitude e fase e identificar as frequências presentes.
Para facilitar a análise, serão utilizados 256 pontos de amostragem e uma frequência de amostragem de 256 Hz.

22.1 Implementação

Código da simulação:

```
# ============================================================
# IMPORTAÇÕES E CONFIGURAÇÕES
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuração visual dos gráficos
sns.set_theme(
    style="whitegrid",
    context="notebook"
)

# ============================================================
# DEFINIÇÃO DOS PARÂMETROS DO SINAL
# ============================================================

fs = 256                # Frequência de amostragem (Hz)
N = 256                 # Número de amostras

A1 = 1.0                # Amplitude da primeira senoide
A2 = 0.5                # Amplitude da segunda senoide

f1 = 10                 # Frequência da primeira senoide (Hz)
f2 = 30                 # Frequência da segunda senoide (Hz)

# Índices das amostras
n = np.arange(N)

# Vetor de tempo
t = n / fs

# ============================================================
# GERAÇÃO DO SINAL NO DOMÍNIO DO TEMPO
# ============================================================

x1 = A1 * np.cos(2 * np.pi * f1 * t)
x2 = A2 * np.cos(2 * np.pi * f2 * t)

# Sinal resultante da soma das duas componentes
x = x1 + x2

# ============================================================
# CÁLCULO DA TRANSFORMADA DISCRETA DE FOURIER
# ============================================================

X = np.fft.fft(x)

# Frequências correspondentes aos coeficientes
frequencias = np.fft.fftfreq(N, d=1/fs)

# ============================================================
# CÁLCULO DO ESPECTRO DE MAGNITUDE
# ============================================================

# Considera apenas as frequências positivas
X_positivo = X[:N//2 + 1]

frequencias_positivas = np.fft.rfftfreq(N, d=1/fs)

# Normalização da magnitude
magnitude = np.abs(X_positivo) / N

# Correção da amplitude para o espectro unilateral
magnitude[1: -1] *= 2

# ============================================================
# CÁLCULO DO ESPECTRO DE FASE
# ============================================================
```

```
fase = np.angle(X_positivo)

# Converte a fase de radianos para graus
fase_graus = np.degrees(fase)

# ============================================================
# IDENTIFICAÇÃO DAS FREQUÊNCIAS
# ============================================================

# Identifica os índices dos maiores picos
indices_picos = np.argsort(magnitude[1:])[ -2:] + 1

# Organiza os picos em ordem crescente de frequência
indices_picos = indices_picos[
    np.argsort(frequencias_positivas[indices_picos])
]

print("FREQUÊNCIAS IDENTIFICADAS")
print(" - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -")

for indice in indices_picos:
    print(
        f"Frequência: {frequencias_positivas[indice]:.2f} Hz | "
        f"Magnitude: {magnitude[indice]:.2f} | "
        f"Fase: {fase_graus[indice]:.2f} graus"
    )

# ============================================================
# GRÁFICO 1 - SINAL NO DOMÍNIO DO TEMPO
# ============================================================

plt.figure(figsize=(12, 5))

plt.plot(
    t,
    x,
    label="Sinal resultante",
    linewidth=1.5
)

plt.plot(
    t,
    x1,
    linestyle=" - -",
    alpha=0.7,
    label="Componente de 10 Hz"
)

plt.plot(
    t,
    x2,
    linestyle=" - -",
    alpha=0.7,
    label="Componente de 30 Hz"
)

plt.title("Sinal composto no domínio do tempo")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.tight_layout()
plt.show()

# ============================================================
# GRÁFICO 2 - ESPECTRO DE MAGNITUDE
# ============================================================

plt.figure(figsize=(12, 5))

plt.stem(
    frequencias_positivas,
    magnitude,
    basefmt=" "
)
```

```
plt.title("Espectro de magnitude da DFT")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Magnitude")
plt.xlim(0, fs/2)

plt.tight_layout()
plt.show()

# ============================================================
# GRÁFICO 3 - ESPECTRO DE FASE
# ============================================================

plt.figure(figsize=(12, 5))

plt.stem(
    frequencias_positivas,
    fase_graus,
    basefmt=" "
)

plt.title("Espectro de fase da DFT")
plt.xlabel("Frequência (Hz)")
plt.ylabel("Fase (graus)")
plt.xlim(0, fs/2)

plt.tight_layout()
plt.show()

# ============================================================
# VERIFICAÇÃO DA RECONSTRUÇÃO DO SINAL
# ============================================================

# Aplicação da DFT inversa
x_reconstruido = np.fft.ifft(X).real

# Cálculo do erro máximo de reconstrução
erro_maximo = np.max(np.abs(x - x_reconstruido))

print("\nVERIFICAÇÃO DA RECONSTRUÇÃO")
print(" - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -")
print(f"Erro máximo: {erro_maximo:.10e}")

# ============================================================
# GRÁFICO 4 - COMPARAÇÃO ENTRE SINAL ORIGINAL E RECONSTRUÍDO
# ============================================================

plt.figure(figsize=(12, 5))

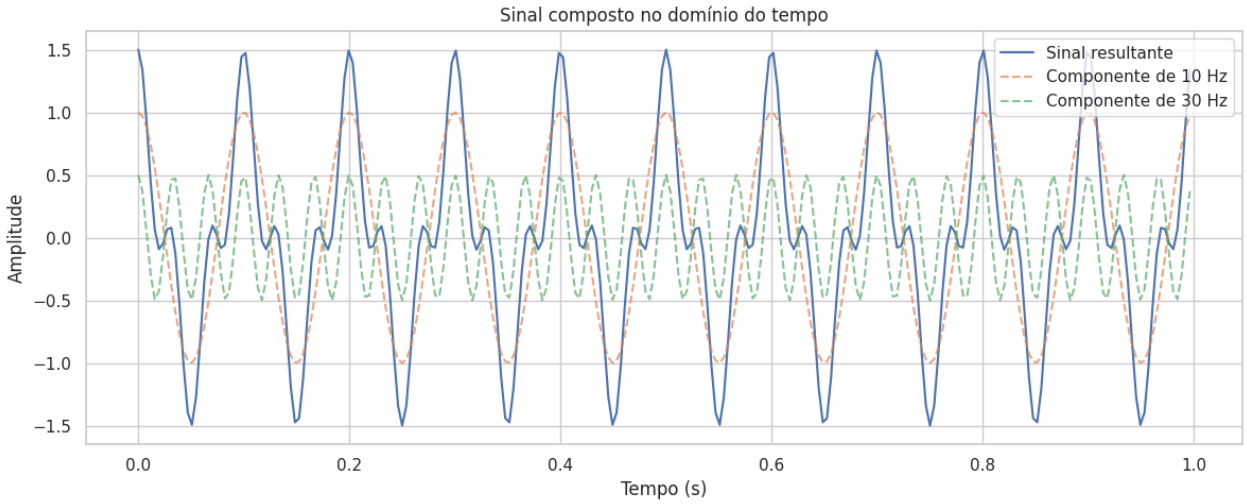
plt.plot(
    t,
    x,
    label="Sinal original",
    linewidth=2
)

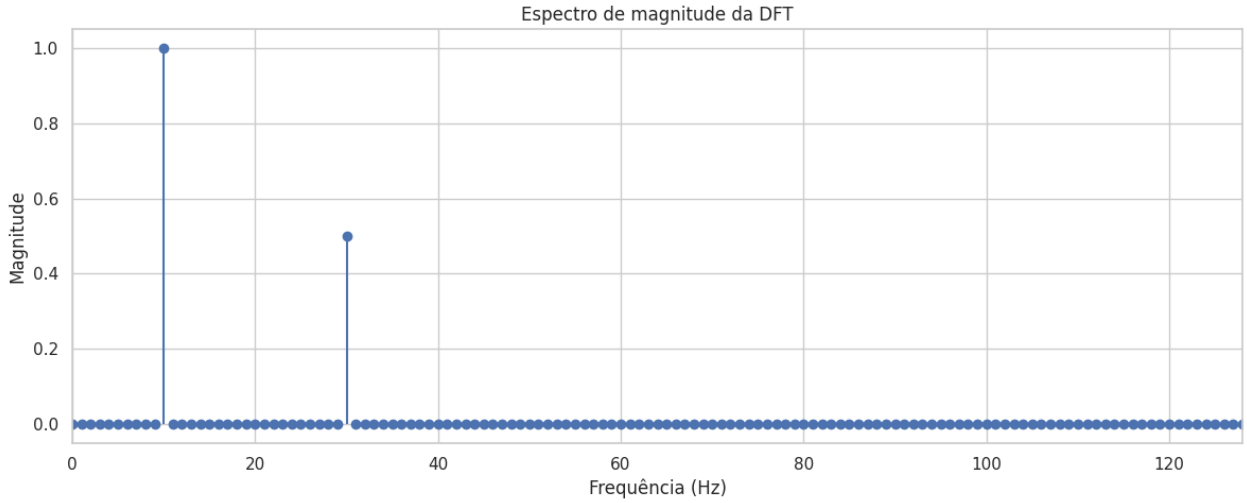
plt.plot(
    t,
    x_reconstruido,
    " - -",
    label="Sinal reconstruído",
    linewidth=1.5
)

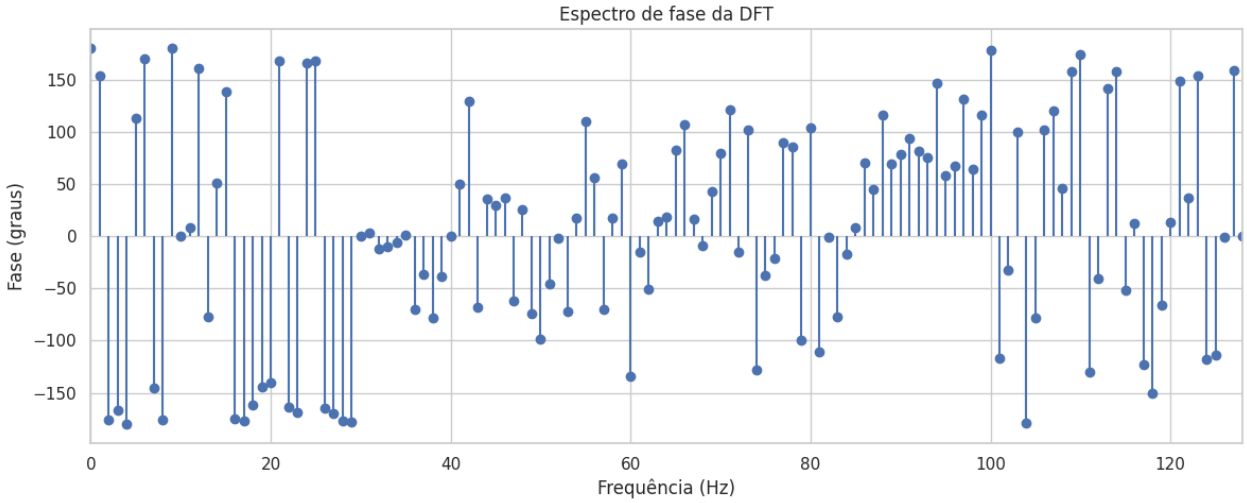
plt.title("Reconstrução do sinal utilizando a DFT inversa")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.legend()

plt.tight_layout()
plt.show()

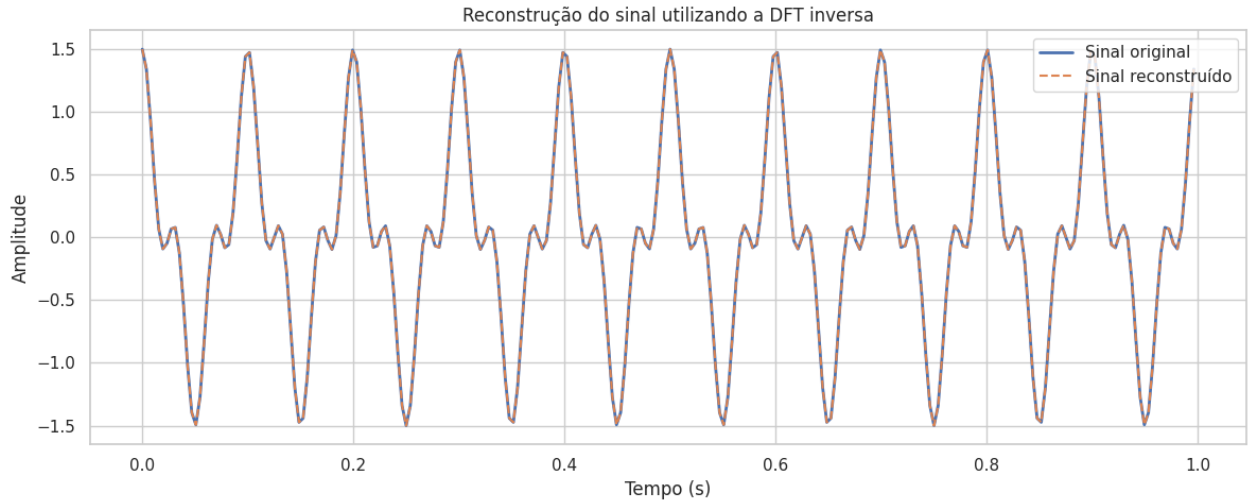
FREQUÊNCIAS IDENTIFICADAS
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Frequência: 10.00 Hz | Magnitude: 1.00 | Fase: -0.00 graus
Frequência: 30.00 Hz | Magnitude: 0.50 | Fase: -0.00 graus
```








```
VERIFICAÇÃO DA RECONSTRUÇÃO
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Erro máximo: 4.4408920985e -16
```



22.2 Etapas da implementação

O código foi dividido em etapas para facilitar a compreensão do processamento realizado. Inicialmente, são definidos os parâmetros das duas senoides, a frequência de amostragem e a quantidade de amostras. Em seguida, são geradas as componentes individuais e realizada a soma para formar o sinal resultante.
Na etapa seguinte, utiliza-se a função `np.fft.fft()` para calcular a DFT do sinal. Essa função implementa a Transformada Rápida de Fourier (FFT), um algoritmo computacionalmente eficiente para calcular a DFT.
Após o cálculo, são obtidos os coeficientes correspondentes às frequências positivas. A magnitude é normalizada pelo número de amostras, com correção dos componentes internos para representar adequadamente as amplitudes do sinal original.
A fase é calculada por meio da função `np.angle()`, sendo posteriormente convertida de radianos para graus.
Por fim, são identificados os dois maiores picos do espectro e realizada a reconstrução do sinal por meio de `np.fft.ifft()`, permitindo verificar a correspondência entre a sequência original e a reconstruída.

---

23 Resultados gráficos

A execução da simulação permite obter quatro representações gráficas distintas, cada uma responsável por apresentar uma característica específica do processamento realizado.

23.1 Sinal no domínio do tempo

O primeiro gráfico apresenta o sinal resultante da soma das duas senoides, juntamente com suas componentes individuais.
É possível observar que o sinal resultante apresenta um comportamento oscilatório formado pela combinação das frequências de 10 Hz e 30 Hz.
A componente de 10 Hz possui amplitude maior, enquanto a componente de 30 Hz apresenta amplitude correspondente à metade da primeira.

23.2 Espectro de magnitude

O segundo gráfico apresenta a magnitude dos coeficientes da DFT em função da frequência. Espera-se observar dois picos principais:

- 10 Hz, com magnitude igual a $1{,}0$;

- 30 Hz, com magnitude igual a $0{,}5$.

Esses picos representam as duas componentes utilizadas na geração do sinal. A ausência de componentes significativas em outras frequências demonstra que, para essa configuração, a DFT identifica corretamente as frequências presentes no sinal.

23.3 Espectro de fase

O terceiro gráfico apresenta a fase dos coeficientes espectrais em função da frequência.
Como as duas senoides foram geradas sem deslocamento de fase, os coeficientes correspondentes às frequências de 10 Hz e 30 Hz apresentam fase inicial igual a zero grau.
É importante observar que os coeficientes com magnitude nula ou muito próxima de zero podem apresentar valores de fase sem significado físico relevante, pois a fase de um componente inexistente não possui interpretação útil.

23.4 Reconstrução do sinal

O quarto gráfico compara o sinal original com aquele reconstruído por meio da DFT inversa.
Espera-se que as duas curvas apresentem sobreposição praticamente perfeita, demonstrando que a transformação preserva as informações necessárias para reconstruir a sequência original.

---

24 Interpretação

Os resultados obtidos demonstram a capacidade da Transformada Discreta de Fourier de identificar as componentes de frequência presentes em um sinal discreto.
No domínio do tempo, o sinal apresenta uma forma de onda resultante da combinação de duas senoides. Entretanto, apenas pela observação temporal, pode ser mais difícil determinar com precisão quais frequências compõem o sinal.
No domínio da frequência, essa identificação torna-se mais simples, pois cada componente aparece representada por um pico na posição correspondente à sua frequência.
Para a simulação realizada, a frequência de amostragem foi de 256 Hz e foram utilizadas 256 amostras. Dessa forma, a resolução em frequência é:

$$\Delta f = \frac{f_s}{N} \tag{45}$$

$$\Delta f = \frac{256}{256} = 1\,\text{Hz} \tag{46}$$

Isso significa que os coeficientes espectrais estão separados por intervalos de 1 Hz.
As frequências utilizadas na geração do sinal, 10 Hz e 30 Hz, coincidem exatamente com índices da DFT, facilitando sua identificação no espectro.
Os resultados esperados podem ser organizados na seguinte tabela:

Componente	Frequência	Amplitude esperada	Fase inicial
Primeira senoide	10 Hz	$1{,}0$	0°
Segunda senoide	30 Hz	$0{,}5$	0°

A magnitude do primeiro pico é duas vezes maior que a do segundo, reproduzindo a proporção entre as amplitudes utilizadas na geração do sinal.
Outro resultado importante é a reconstrução da sequência por meio da DFT inversa. Como a transformação preserva as informações de magnitude e fase dos coeficientes, a sequência original pode ser recuperada com erro numérico muito pequeno, decorrente apenas da precisão computacional.
Portanto, a simulação confirma a relação entre os componentes utilizados na geração do sinal e os picos observados no espectro de magnitude.

25 Discussão crítica

A Transformada Discreta de Fourier é uma ferramenta fundamental para a análise de sinais digitais, pois permite identificar componentes de frequência que nem sempre são facilmente reconhecidas pela observação direta do sinal no domínio do tempo.
Uma de suas principais vantagens é a possibilidade de decompor matematicamente uma sequência em componentes espectrais, fornecendo informações sobre frequência, magnitude e fase. Além disso, a utilização da FFT torna o cálculo computacionalmente eficiente, permitindo analisar sequências com grandes quantidades de amostras.
Entretanto, a aplicação da DFT apresenta algumas limitações que precisam ser consideradas. A primeira está relacionada à resolução em frequência, que depende diretamente da quantidade de amostras e da frequência de amostragem. Quando a resolução é insuficiente, componentes de frequência próximas podem apresentar dificuldade de distinção.
Outra limitação está relacionada ao fenômeno de vazamento espectral (*spectral leakage*), que ocorre quando o sinal analisado não apresenta um número inteiro de ciclos dentro do intervalo observado. Nessa situação, a energia de uma componente pode se distribuir entre diferentes pontos do espectro, dificultando a identificação precisa de sua frequência e magnitude.
Além disso, a frequência de amostragem deve respeitar o critério de Nyquist, evitando que componentes acima da metade da frequência de amostragem sejam representadas incorretamente no espectro.
Na simulação apresentada, essas dificuldades foram minimizadas pela escolha de frequências que coincidem exatamente com os pontos da DFT, utilizando uma quantidade de amostras e uma frequência de amostragem que proporcionam resolução de 1 Hz.
Apesar disso, em aplicações reais, os sinais podem apresentar ruídos, variações de amplitude, frequências não inteiras em relação à resolução espectral e componentes simultâneas. Nesses casos, podem ser necessárias técnicas complementares, como a aplicação de janelas, o aumento do número de amostras e métodos de estimação espectral.
Conclui-se que a DFT é essencial para a análise de sinais no domínio da frequência, permitindo identificar e quantificar suas componentes espectrais. A simulação demonstrou que os picos encontrados correspondem às frequências utilizadas na geração do sinal e que a aplicação da transformada inversa permite recuperar a sequência original. Dessa maneira, a atividade evidencia a relação entre a representação temporal e espectral de um sinal, demonstrando a importância da DFT no Processamento Digital de Sinais.

26 Transformada Rápida de Fourier (FFT)

27 Resumo conceitual

A FFT é uma família de algoritmos eficientes para calcular a DFT. O resultado matemático é o mesmo da soma direta; a diferença está na quantidade de operações.

28 Formulação matemática

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}. \tag{47}$$

A soma direta exige ordem N^2 de operações; algoritmos FFT adequados exigem ordem $N \log_2 N$. O erro comparado é $\max_k |X_{direta}[k] - X_{FFT}[k]|$.

29 Exemplo

Compare N = 32, 64, 128 e 256 para uma senoide de 5 ciclos por registro. Os tempos são medianas de cinco execuções e dependem da máquina.

---

30 Simulação

Compare N = 32, 64, 128 e 256 para uma senoide de 5 ciclos por registro. Os tempos são medianas de cinco execuções e dependem da máquina.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

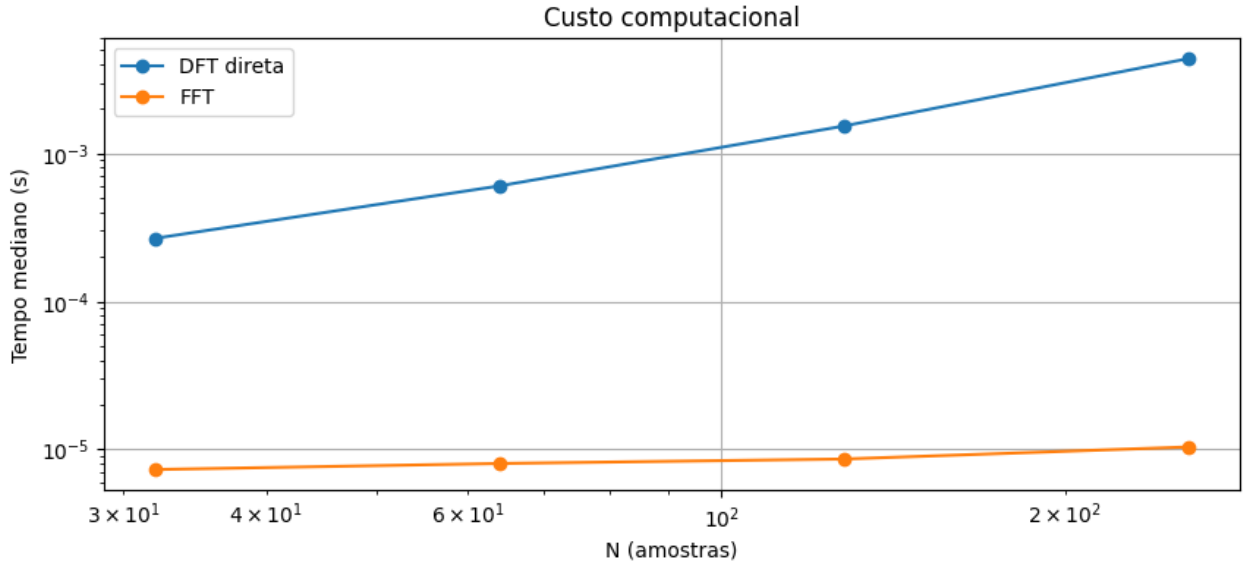
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))
```

```
from time import perf_counter
linhas=[]
for N in [32,64,128,256]:
    n=np.arange(N); x=np.cos(2*np.pi*5*n/N)
    def direta(x):
        return np.array([np.sum(x*np.exp( -2j*np.pi*k*n/N)) for k in range(N)])
    tempos=[]
    for metodo in [direta,np.fft.fft]:
        medicoes=[]
        for _ in range(5):
            inicio=perf_counter(); X=metodo(x); medicoes.append(perf_counter() -inicio)
        tempos.append(np.median(medicoes))
    erro=np.max(np.abs(direta(x) -np.fft.fft(x)))
    assert erro < 1e -9
    linhas.append([N,*tempos,erro])
df=pd.DataFrame(linhas,columns=['N','DFT (s)','FFT (s)','Erro máximo']); display(df)
plt.loglog(df.N,df['DFT (s)'],'o -',label='DFT direta'); plt.loglog(df.N,df['FFT (s)'],'o -',label='FFT')
plt.xlabel('N (amostras)'); plt.ylabel('Tempo mediano (s)'); plt.title('Custo computacional'); plt.legend();
     plt.show()
```

	N	DFT (s)	FFT (s)	Erro máximo
0	32	0.000267	0.000007	1.139129e-13
1	64	0.000604	0.000008	2.993740e-13
2	128	0.001531	0.000009	1.346403e-12
3	256	0.004367	0.000010	4.066505e-12



31 Resultados

A tabela apresenta o erro numérico e os tempos medidos. As duas implementações produzem coeficientes equivalentes dentro da tolerância de 10^{-9}.

32 Interpretação e Discussão

A FFT reduz o custo de calcular o mesmo espectro. Para registros pequenos, overhead e implementação em Python afetam a comparação; os tempos medidos não isolam apenas a complexidade assintótica.

33 Conclusões

A tabela apresenta o erro numérico e os tempos medidos. As duas implementações produzem coeficientes equivalentes dentro da tolerância de 10^{-9}.

34 Resolução Espectral

---

35 Resumo conceitual

A duração de observação limita a separação de frequências próximas. O preenchimento com zeros aumenta a densidade do gráfico, mas não acrescenta informação ao registro.

---

36 Formulação matemática

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_{obs}}. \tag{48}$$

Aqui $T_{obs} = N/f_s$ é a duração do registro. A resolução efetiva depende também da largura do lóbulo principal da janela.

---

37 Exemplo

Use fs = 1024 Hz, frequências de 100 e 108 Hz e N = 64, 256 e 1024. Compare ainda N = 64 preenchido com zeros até 1024.

---

38 Simulação

Use fs = 1024 Hz, frequências de 100 e 108 Hz e N = 64, 256 e 1024. Compare ainda N = 64 preenchido com zeros até 1024.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

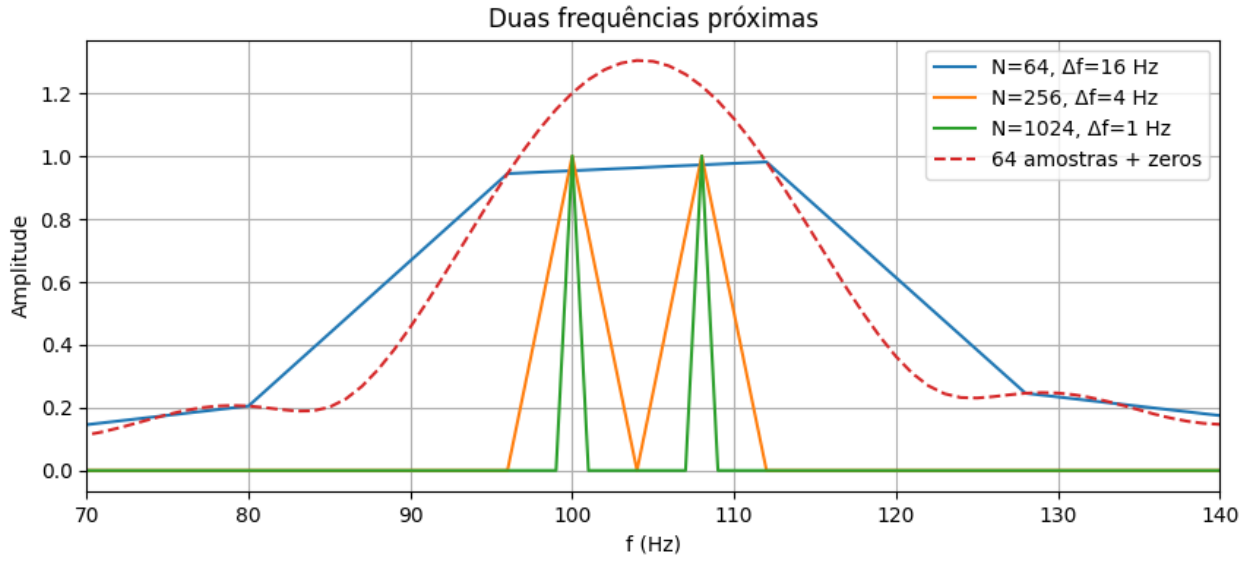
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A
```

```
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1024
for N in [64,256,1024]:
    t=np.arange(N)/fs; x=np.cos(2*np.pi*100*t)+np.cos(2*np.pi*108*t)
    f,A=espectro(x,fs); plt.plot(f,A,label=f'N={N}, \Deltaf={fs/N:g} Hz')
    print(N,'Tobs =',N/fs,'s; \Deltaf =',fs/N,'Hz')
t=np.arange(64)/fs; x=np.cos(2*np.pi*100*t)+np.cos(2*np.pi*108*t)
f,A=espectro(x,fs,nfft=1024); plt.plot(f,A,' - -',label='64 amostras + zeros')
plt.xlim(70,140); plt.xlabel('f (Hz)'); plt.ylabel('Amplitude'); plt.title('Duas frequências próximas'); plt
    .legend(); plt.show()

64 Tobs = 0.0625 s; \Deltaf = 16.0 Hz
256 Tobs = 0.25 s; \Deltaf = 4.0 Hz
1024 Tobs = 1.0 s; \Deltaf = 1.0 Hz
```



39 Resultados

Os espaçamentos são 16, 4 e 1 Hz. O registro de 1024 amostras separa claramente as componentes de 100 e 108 Hz.

40 Interpretação e Discussão

O registro curto não oferece a mesma separação mesmo após o preenchimento com zeros. Aumentar N melhora a resolução somente quando aumenta o tempo efetivamente observado.

41 Conclusões

Os espaçamentos são 16, 4 e 1 Hz. O registro de 1024 amostras separa claramente as componentes de 100 e 108 Hz.

42 Frequência Normalizada

---

43 Resumo conceitual

A frequência discreta mede ciclos ou radianos por amostra. A conversão para hertz requer conhecer a taxa de amostragem.

---

44 Formulação matemática

$$f_n = f/f_s, \qquad \omega = 2\pi f_n, \qquad f = \omega f_s/(2\pi). \tag{49}$$

Frequências separadas por 2π rad/amostra geram a mesma exponencial complexa discreta.

---

45 Exemplo

Compare (f, fs) = (50, 1000), (100, 1000) e (50, 500), em hertz.

---

46 Simulação

Compare (f, fs) = (50, 1000), (100, 1000) e (50, 500), em hertz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

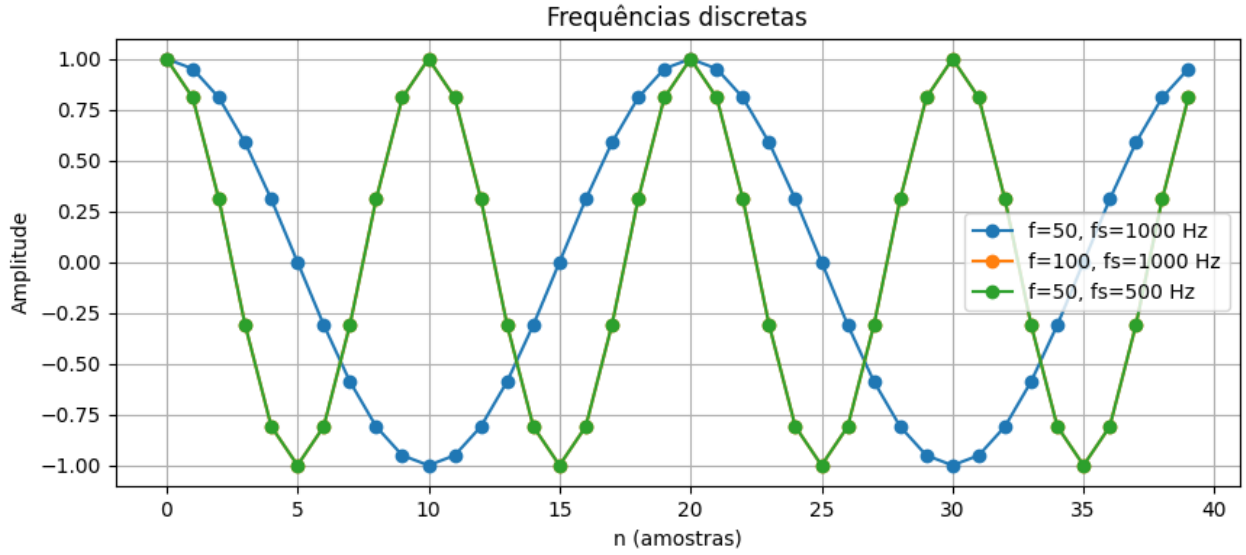
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

linhas=[]
for f,fs in [(50,1000),(100,1000),(50,500)]:
    linhas.append([f,fs,f/fs,2*np.pi*f/fs])
display(pd.DataFrame(linhas,columns=['f (Hz)','fs (Hz)','Ciclos/amostra','rad/amostra']))
n=np.arange(40)
for f,fs in [(50,1000),(100,1000),(50,500)]:
    plt.plot(n,np.cos(2*np.pi*f*n/fs),'o -',label=f'f={f}, fs={fs} Hz')
plt.xlabel('n (amostras)'); plt.ylabel('Amplitude'); plt.title('Frequências discretas'); plt.legend(); plt.
     show()
```

	f (Hz)	fs (Hz)	Ciclos/amostra	rad/amostra
0	50	1000	0.05	0.314159
1	100	1000	0.10	0.628319
2	50	500	0.10	0.628319



---

47 Resultados

As frequências angulares são 0,1π, 0,2π e 0,2π rad/amostra. Os dois últimos sinais coincidem.

48 Interpretação e Discussão

Uma sequência não determina sua frequência física sem fs. Metade da taxa de amostragem corresponde a π rad/amostra e a 0,5 ciclo/amostra.

49 Conclusões

As frequências angulares são 0,1π, 0,2π e 0,2π rad/amostra. Os dois últimos sinais coincidem.

50 Teorema da Amostragem e Nyquist

---

51 Resumo conceitual

A amostragem uniforme de um sinal limitado em banda permite reconstrução ideal quando as réplicas espectrais não se sobrepõem. O limite exato exige cuidado com fase e componentes na borda.

---

52 Formulação matemática

$$f_s > 2f_{max}, \qquad f_N = f_s/2. \tag{50}$$

A reconstrução ideal usa interpolação sinc e pressupõe um sinal limitado em banda e amostragem sem erros.

---

53 Exemplo

Amostre uma senoide de 70 Hz a 500, 140 e 100 Hz. O caso de 140 Hz é o limite e deve ser comparado com uma senoide em quadratura.

---

54 Simulação

Amostre uma senoide de 70 Hz a 500, 140 e 100 Hz. O caso de 140 Hz é o limite e deve ser comparado com uma senoide em quadratura.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

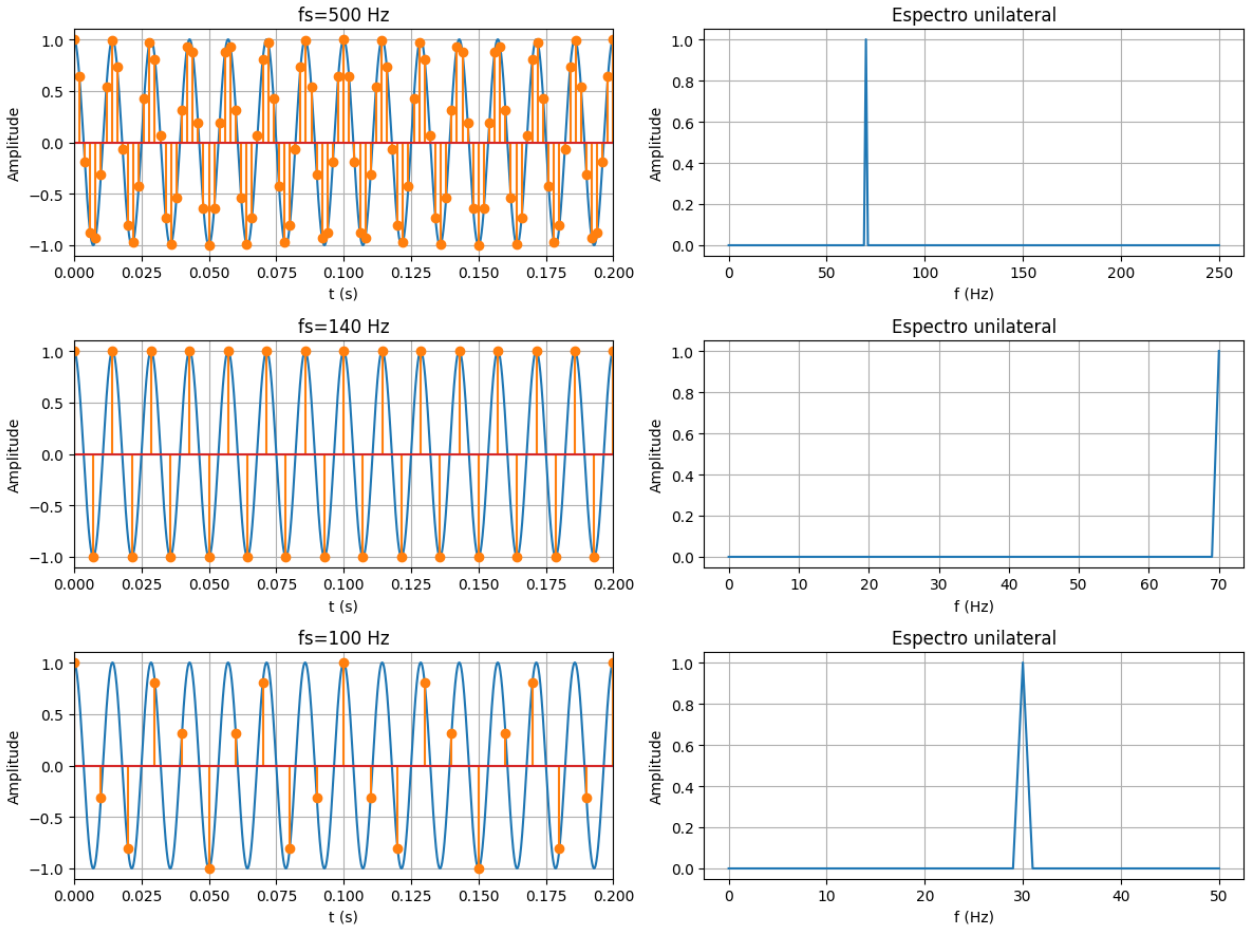
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

f0=70; tc=np.linspace(0,0.2,4000)
fig,ax=plt.subplots(3,2,figsize=(12,9))
for i,fs in enumerate([500,140,100]):
    t=np.arange(fs)/fs; x=np.cos(2*np.pi*f0*t)
    ax[i,0].plot(tc,np.cos(2*np.pi*f0*tc),label='Referência contínua simulada')
    ax[i,0].stem(t[:int(.2*fs)+1],x[:int(.2*fs)+1],linefmt='C1 -',markerfmt='C1o')
```

```
    ax[i,0].set(xlim=(0,.2),title=f'fs={fs} Hz',xlabel='t (s)',ylabel='Amplitude')
    f,A=espectro(x,fs); ax[i,1].plot(f,A); ax[i,1].set(title='Espectro unilateral',xlabel='f (Hz)',ylabel='
     Amplitude')
    print('fs =',fs,'Hz; frequência aparente =',abs((f0+fs/2)%fs -fs/2),'Hz')
plt.tight_layout(); plt.show()
# No limite de Nyquist, uma senoide com fase zero pode desaparecer.
n=np.arange(20); print('Máximo da senoide em Nyquist:',np.max(np.abs(np.sin(np.pi*n))))
```

```
fs = 500 Hz; frequência aparente = 70.0 Hz
fs = 140 Hz; frequência aparente = 70.0 Hz
fs = 100 Hz; frequência aparente = 30.0 Hz
```




```
Máximo da senoide em Nyquist: 5.879542597180472e -15
```

---

55 Resultados

A 500 Hz o pico aparece em 70 Hz; a 100 Hz aparece em 30 Hz. A 140 Hz o cosseno alterna sinais, mas uma senoide de fase zero gera amostras praticamente nulas.

56 Interpretação e Discussão

O limite de Nyquist não garante preservação de uma componente arbitrária exatamente na borda. Sistemas reais necessitam margem para a transição do filtro analógico antialiasing.

57 Conclusões

A 500 Hz o pico aparece em 70 Hz; a 100 Hz aparece em 30 Hz. A 140 Hz o cosseno alterna sinais, mas uma senoide de fase zero gera amostras praticamente nulas.

58 Aliasing

---

59 Resumo conceitual

Aliasing é a ambiguidade entre frequências que produzem as mesmas amostras. Uma vez sobrepostas, essas contribuições não podem ser separadas apenas por filtragem digital posterior.

---

60 Formulação matemática

$$f_a = \left|((f_0 + f_s/2) \bmod f_s) - f_s/2\right|. \tag{51}$$

Para cossenos reais, $\cos(2\pi 70 n/100) = \cos(2\pi 30 n/100)$.

---

61 Exemplo

Compare um cosseno de 70 Hz com outro de 30 Hz, ambos amostrados a 100 Hz.

---

62 Simulação

Compare um cosseno de 70 Hz com outro de 30 Hz, ambos amostrados a 100 Hz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

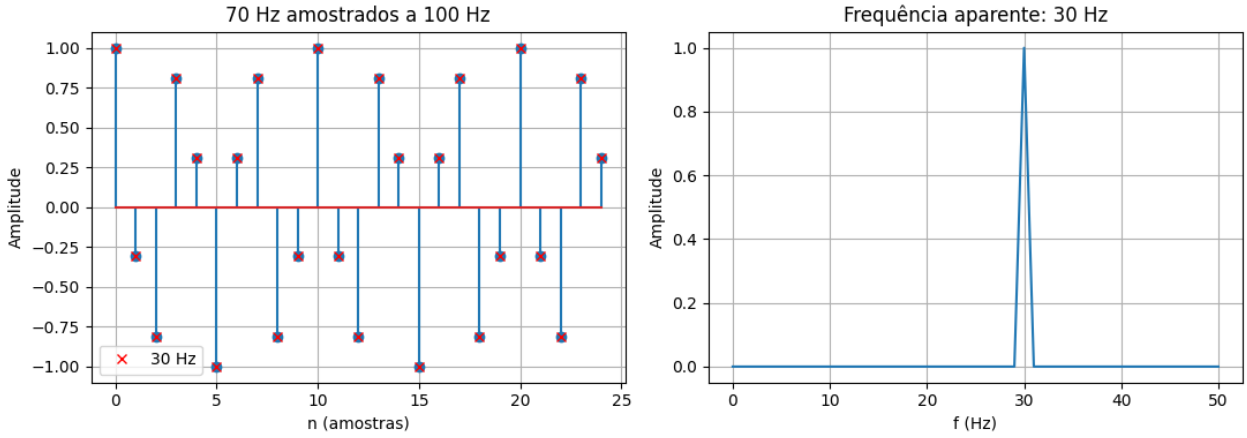
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
```

```
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=100; n=np.arange(100); x=np.cos(2*np.pi*70*n/fs); xa=np.cos(2*np.pi*30*n/fs)
assert np.allclose(x,xa)
print('Erro máximo entre as sequências:',np.max(np.abs(x -xa)))
fig,ax=plt.subplots(1,2,figsize=(11,4))
ax[0].stem(n[:25],x[:25]); ax[0].plot(n[:25],xa[:25],'rx',label='30 Hz'); ax[0].legend(); ax[0].set(xlabel='
    n (amostras)',ylabel='Amplitude',title='70 Hz amostrados a 100 Hz')
f,A=espectro(x,fs); ax[1].plot(f,A); ax[1].set(xlabel='f (Hz)',ylabel='Amplitude',title='Frequência aparente
    : 30 Hz'); plt.tight_layout(); plt.show()

Erro máximo entre as sequências: 1.1091128016005314e -13
```



---

63 Resultados

As sequências coincidem dentro da precisão numérica e o espectro apresenta pico em 30 Hz.

64 Interpretação e Discussão

Em sensores de vibração, aliasing pode ser interpretado como uma oscilação mecânica inexistente. A prevenção exige filtrar antes da aquisição e escolher fs com margem para a banda

do sinal.

65 Conclusões

As sequências coincidem dentro da precisão numérica e o espectro apresenta pico em 30 Hz.

66 Janelamento e Vazamento Espectral

---

67 Resumo conceitual

O recorte temporal multiplica o sinal por uma janela e espalha sua representação espectral. Janelas suaves reduzem lóbulos laterais, mas alargam o lóbulo principal.

---

68 Formulação matemática

$$x_w[n] = x[n]w[n], \qquad X_w(e^{j\omega}) = \frac{1}{2\pi}(X * W)(e^{j\omega}). \tag{52}$$

Para uma senoide isolada longe de DC e Nyquist, a amplitude unilateral é estimada por $2|X_w[k]|/\sum_n w[n]$.

---

69 Exemplo

Use 256 amostras a 1024 Hz de uma senoide de 100,5 Hz e compare retangular, Hann, Hamming e Blackman. O preenchimento com zeros serve apenas para visualizar os lóbulos.

---

70 Simulação

Use 256 amostras a 1024 Hz de uma senoide de 100,5 Hz e compare retangular, Hann, Hamming e Blackman. O preenchimento com zeros serve apenas para visualizar os lóbulos. Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
```

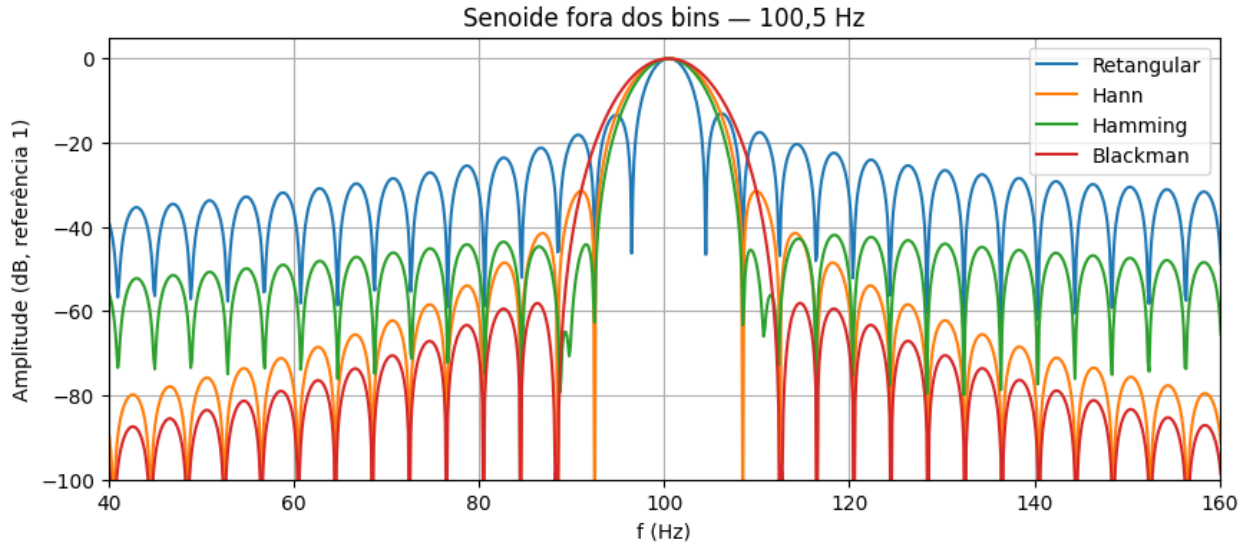
```
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1024; N=256; n=np.arange(N); x=np.cos(2*np.pi*100.5*n/fs)
for nome,w in [('Retangular',np.ones(N)),('Hann',signal.windows.hann(N,sym=False)),('Hamming',signal.windows
    .hamming(N,sym=False)),('Blackman',signal.windows.blackman(N,sym=False))]:
    f,A=espectro(x,fs,w,nfft=8192)
    plt.plot(f,20*np.log10(np.maximum(A,1e -8)),label=nome)
    print(nome,'ganho coerente =',w.mean())
plt.xlim(40,160); plt.ylim( -100,5); plt.xlabel('f (Hz)'); plt.ylabel('Amplitude (dB, referência 1)'); plt.
    title('Senoide fora dos bins — 100,5 Hz'); plt.legend(); plt.show()

Retangular ganho coerente = 1.0
Hann ganho coerente = 0.5
Hamming ganho coerente = 0.54
Blackman ganho coerente = 0.42
```



71 Resultados

A janela retangular tem lóbulo principal estreito e lóbulos laterais elevados. Hann e Hamming reduzem os lóbulos laterais; Blackman amplia mais o lóbulo principal.

72 Interpretação e Discussão

O ganho coerente deve ser compensado para comparar amplitudes. Componentes próximas podem se fundir após o janelamento; menor vazamento não significa automaticamente maior capacidade de separação.

73 Conclusões

A janela retangular tem lóbulo principal estreito e lóbulos laterais elevados. Hann e Hamming reduzem os lóbulos laterais; Blackman amplia mais o lóbulo principal.

74 Transformada de Laplace

---

75 Resumo conceitual

A transformada de Laplace representa sinais contínuos por exponenciais complexas. A parte real de s controla crescimento ou decaimento e a parte imaginária descreve oscilação.

---

76 Formulação matemática

$$X(s) = \int_{-\infty}^{\infty} x(t) e^{-st}\, dt, \quad s = \sigma + j\Omega. \tag{53}$$

Para $e^{-at}u(t)$, $X(s) = 1/(s+a)$ e Re $s > -a$. Para $e^{-at}\sin(bt)u(t)$, $X(s) = b/((s+a)^2 + b^2)$, na mesma ROC.

---

77 Exemplo

Analise $e^{-2t}u(t)$ e $e^{-t}\text{sen}(4t)u(t)$. A primeira integral é $\int_0\infty e^{-(s.2)t}dt = 1/(s+2)$; a segunda decorre da decomposição do seno em duas exponenciais.

---

78 Simulação

Analise $e^{-2t}u(t)$ e $e^{-t}\text{sen}(4t)u(t)$. A primeira integral é $\int_0\infty e^{-(s.2)t}dt = 1/(s+2)$; a segunda decorre da decomposição do seno em duas exponenciais.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

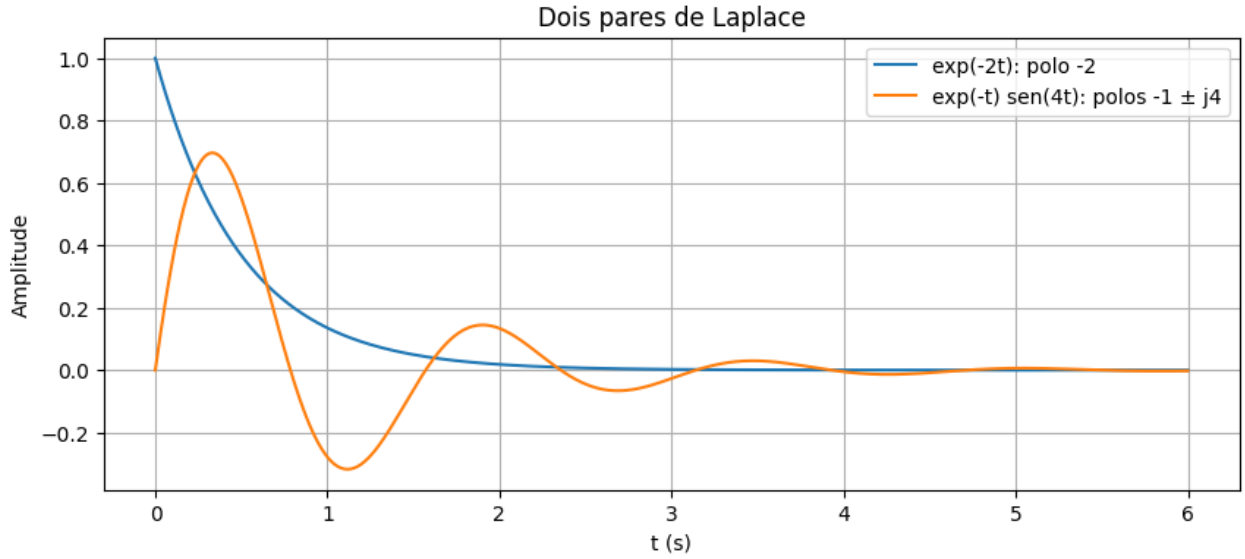
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

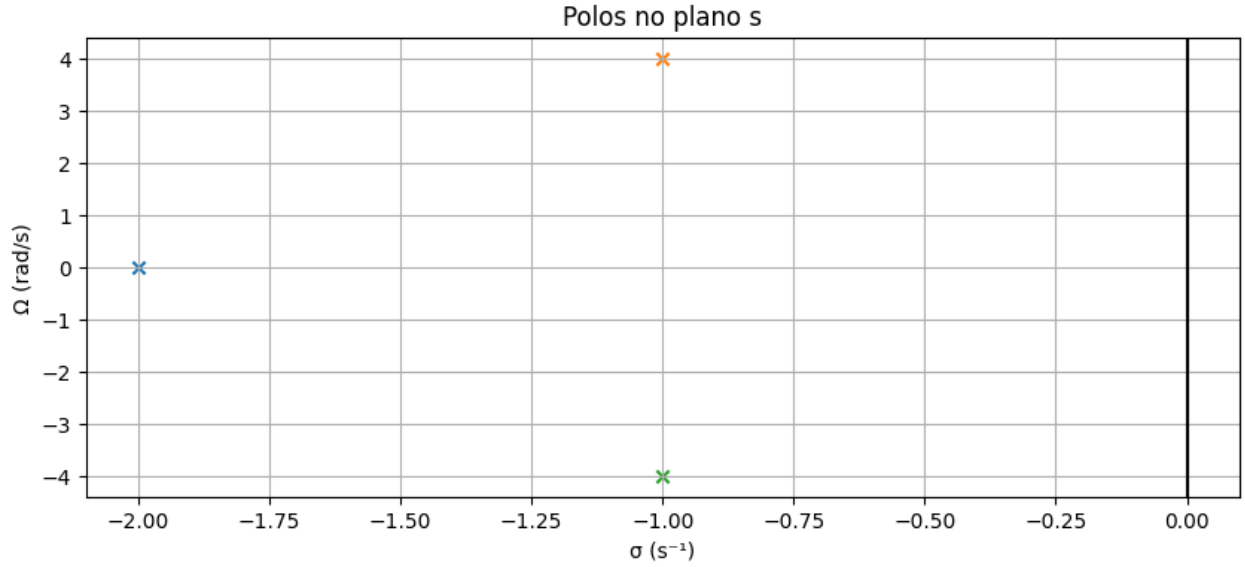
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

t=np.linspace(0,6,1200)
plt.plot(t,np.exp( -2*t),label='exp( -2t): polo -2')
plt.plot(t,np.exp( -t)*np.sin(4*t),label='exp( -t) sen(4t): polos -1 \pm j4')
plt.xlabel('t (s)'); plt.ylabel('Amplitude'); plt.title('Dois pares de Laplace'); plt.legend(); plt.show()
for p in [ -2, -1+4j, -1 -4j]: plt.scatter(np.real(p),np.imag(p),marker='x')
plt.axvline(0,color='k'); plt.xlabel(' \sigma (⁻s¹)'); plt.ylabel(' \Omega (rad/s)'); plt.title('Polos no
     plano s'); plt.show()
```





---

79 Resultados

A primeira sequência decai sem oscilar, com constante de tempo 0,5 s. A segunda oscila a 4 rad/s sob envoltória de constante de tempo 1 s.

80 Interpretação e Discussão

Os polos explicam simultaneamente velocidade de decaimento e oscilação. A expressão racional precisa da ROC para determinar o sinal bilateral; as hipóteses causais são explicitadas nos exemplos.

81 Conclusões

A primeira sequência decai sem oscilar, com constante de tempo 0,5 s. A segunda oscila a 4 rad/s sob envoltória de constante de tempo 1 s.

82 Função de Transferência Contínua

---

83 Resumo conceitual

A função de transferência relaciona saída e entrada de um sistema LTI sob condições iniciais nulas. Ganho, constante de tempo, amortecimento e frequência natural controlam a resposta temporal.

---

84 Formulação matemática

$$H(s) = Y(s)/X(s), \quad H_1(s) = K/(\tau s + 1), \quad H_2(s) = \omega_n^2/(s^2 + 2\zeta\omega_n s + \omega_n^2). \tag{54}$$

No primeiro caso, $\tau\dot{y} + y = Kx$ e a resposta ao degrau unitário é $K(1 - e^{-t/\tau})u(t)$.

---

85 Exemplo

Compare K = 1 e 2; τ = 0,2 e 1 s; ζ = 0,2, 0,7 e 1,2; ωn = 4 e 8 rad/s.

---

86 Simulação

Compare K = 1 e 2; τ = 0,2 e 1 s; ζ = 0,2, 0,7 e 1,2; ωn = 4 e 8 rad/s.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

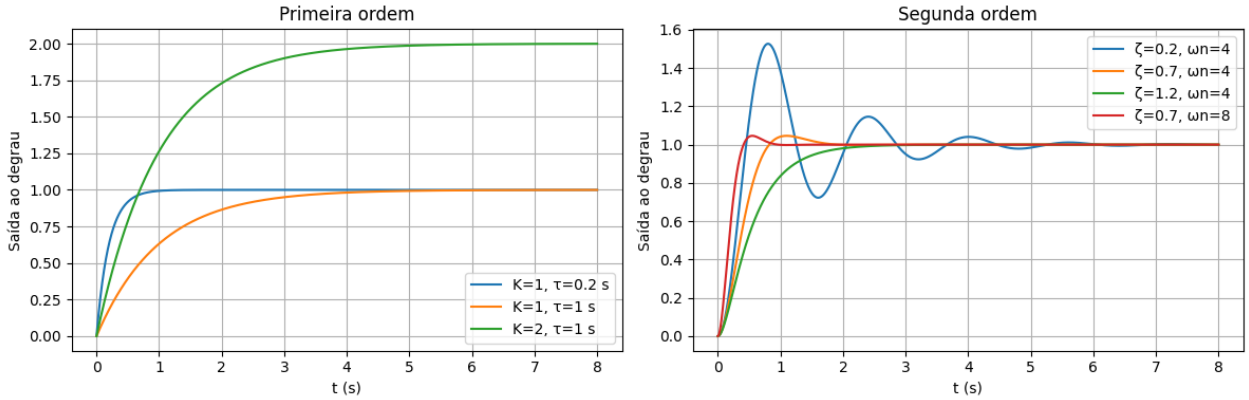
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
```

```
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

t=np.linspace(0,8,1600); fig,ax=plt.subplots(1,2,figsize=(12,4))
for K,tau in [(1,.2),(1,1),(2,1)]:
    _,y=signal.step(([K],[tau,1]),T=t); ax[0].plot(t,y,label=f'K={K}, \tau={tau} s')
for zeta,wn in [(.2,4),(.7,4),(1.2,4),(.7,8)]:
    _,y=signal.step(([wn**2],[1,2*zeta*wn,wn**2]),T=t); ax[1].plot(t,y,label=f' \zeta={zeta}, \omegan={wn}')
for a in ax: a.set(xlabel='t (s)',ylabel='Saída ao degrau'); a.legend()
ax[0].set_title('Primeira ordem'); ax[1].set_title('Segunda ordem'); plt.tight_layout(); plt.show()
```



---

87 Resultados

O ganho determina o valor final da primeira ordem. Menor τ acelera a resposta. Baixo amortecimento provoca oscilação e sobressinal; maior ωn acelera a dinâmica.

88 Interpretação e Discussão

A função de transferência não inclui a resposta livre produzida por condições iniciais não nulas. Comparar sistemas exige declarar tanto a entrada quanto as condições iniciais.

89 Conclusões

O ganho determina o valor final da primeira ordem. Menor τ acelera a resposta. Baixo amortecimento provoca oscilação e sobressinal; maior ωn acelera a dinâmica.

90 Polos, Zeros e Estabilidade no Plano s

---

91 Resumo conceitual

Para sistemas contínuos causais racionais próprios, polos de transferência no semiplano esquerdo garantem estabilidade BIBO. Polos complexos produzem modos oscilatórios.

---

92 Formulação matemática

$$H(s) = K\frac{\prod_i (s - z_i)}{\prod_k (s - p_k)}, \qquad h_p(t) = e^{pt}u(t). \tag{55}$$

Um polo $p = \sigma + j\Omega$ produz envoltória $e^{\sigma t}$ e oscilação de frequência angular Ω.

---

93 Exemplo

Compare polos -1, -5, +1 e -1 ± j4. Inclua um zero em -2 no sistema oscilatório para observar sua influência na resposta.

---

94 Simulação

Compare polos -1, -5, +1 e -1 ± j4. Inclua um zero em -2 no sistema oscilatório para observar sua influência na resposta.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

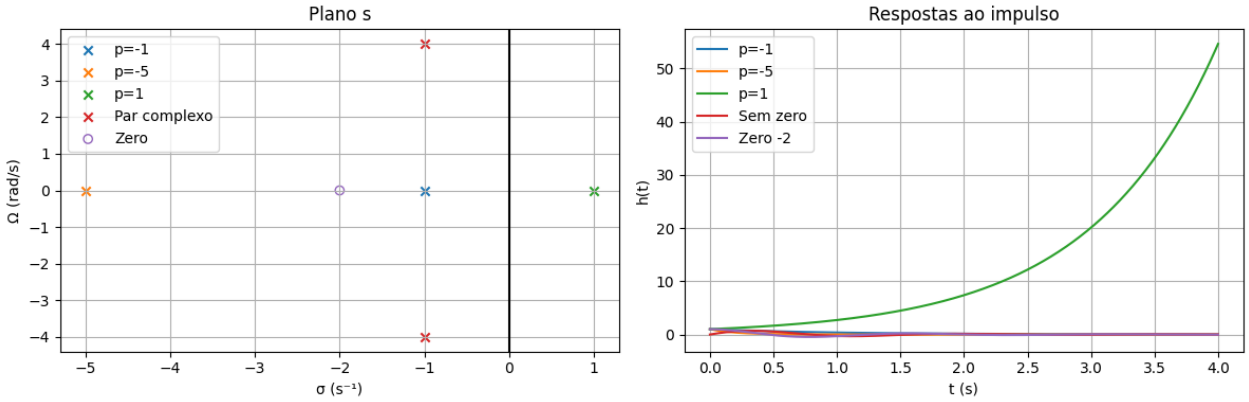
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
```

```
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

t=np.linspace(0,4,800); fig,ax=plt.subplots(1,2,figsize=(12,4))
for p in [ -1, -5,1]:
    ax[0].scatter(p,0,marker='x',label=f'p={p}'); ax[1].plot(t,np.exp(p*t),label=f'p={p}')
for b,label in [([4],'Sem zero'),([1,2],'Zero -2')]:
    _,h=signal.impulse((b,[1,2,17]),T=t); ax[1].plot(t,h,label=label)
ax[0].scatter([ -1, -1],[4, -4],marker='x',label='Par complexo'); ax[0].scatter( -2,0,facecolors='none',
     edgecolors='C4',label='Zero')
ax[0].axvline(0,color='k'); ax[0].set(xlabel=' \sigma (⁻s¹)',ylabel=' \Omega (rad/s)',title='Plano s')
ax[1].set(xlabel='t (s)',ylabel='h(t)',title='Respostas ao impulso')
for a in ax: a.legend()
plt.tight_layout(); plt.show()
```



---

95 Resultados

O polo -5 decai mais rápido que -1. O polo +1 cresce. O par complexo produz oscilação amortecida e o zero modifica a forma da resposta.

96 Interpretação e Discussão

O critério se refere aos polos da função de transferência sem cancelamentos ocultos. Estabilidade interna exige analisar também os modos do modelo de estados.

97 Conclusões

O polo -5 decai mais rápido que -1. O polo +1 cresce. O par complexo produz oscilação amortecida e o zero modifica a forma da resposta.

98 Transformada Z

---

99 Resumo conceitual

A transformada Z estende a análise de Fourier para além do círculo unitário. Ela converte atrasos e equações de diferenças em relações algébricas.

---

100 Formulação matemática

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}, \quad z = re^{j\omega}. \tag{56}$$

Pares elementares: $\delta[n] \leftrightarrow 1$; $u[n] \leftrightarrow 1/(1 - z^{-1})$, $|z| > 1$; $a^n u[n] \leftrightarrow 1/(1 - az^{-1})$, $|z| > |a|$.

---

101 Exemplo

Use a = 0,8 e compare a soma finita de 100 termos com a fórmula analítica em z = 1,2e^{jw}.

---

102 Simulação

Use a = 0,8 e compare a soma finita de 100 termos com a fórmula analítica em z = 1,2e^{jw}. Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

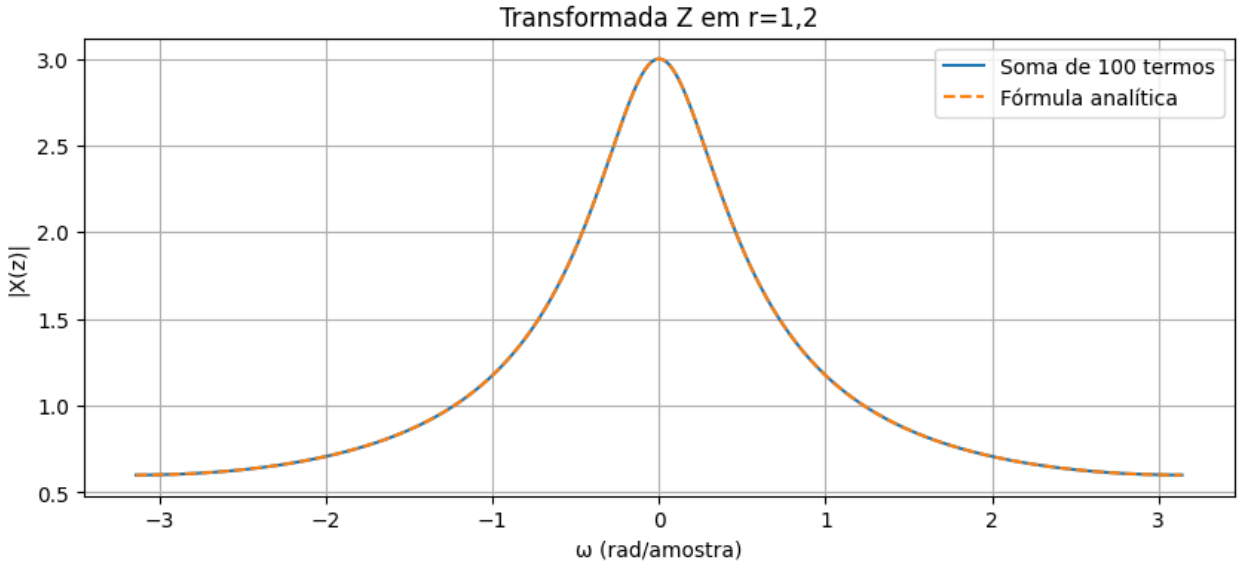
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
```

```
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

a=.8; n=np.arange(100); w=np.linspace( -np.pi,np.pi,600); z=1.2*np.exp(1j*w)
X=(a**n)@z[:,None].__pow__( -n).T
Xa=1/(1 -a/z)
print('Erro máximo da soma truncada:',np.max(np.abs(X -Xa)))
plt.plot(w,np.abs(X),label='Soma de 100 termos'); plt.plot(w,np.abs(Xa),' - -',label='Fórmula analítica')
plt.xlabel(' \omega (rad/amostra)'); plt.ylabel('|X(z)|'); plt.title('Transformada Z em r=1,2'); plt.legend
    (); plt.show()
```

```
Erro máximo da soma truncada: 2.220446049250313e -15
```



---

103 Resultados

A soma truncada aproxima a expressão racional porque |a/z| <1 em todos os pontos avaliados.

104 Interpretação e Discussão

A convergência depende do raio r. Um valor finito calculado pela expressão racional fora da ROC é uma continuação algébrica e não prova convergência da série.

105 Conclusões

A soma truncada aproxima a expressão racional porque |a/z| <1 em todos os pontos avaliados.

106 Região de Convergência (ROC)

---

107 Resumo conceitual

A ROC faz parte da definição da transformada Z. A mesma expressão racional pode representar uma sequência causal ou anticausal, conforme a região escolhida.

---

108 Formulação matemática

$$\frac{1}{1-az^{-1}} \leftrightarrow a^n u[n],\ |z|>|a|; \qquad \frac{1}{1-az^{-1}} \leftrightarrow -a^n u[-n-1],\ |z|<|a|. \tag{57}$$

A estabilidade exige que a ROC inclua o círculo unitário.

---

109 Exemplo

Para a = 0,8, a sequência causal é estável; a anticausal cresce para n negativo e sua ROC não contém |z| = 1.

---

110 Simulação

Para a = 0,8, a sequência causal é estável; a anticausal cresce para n negativo e sua ROC não contém |z| = 1.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

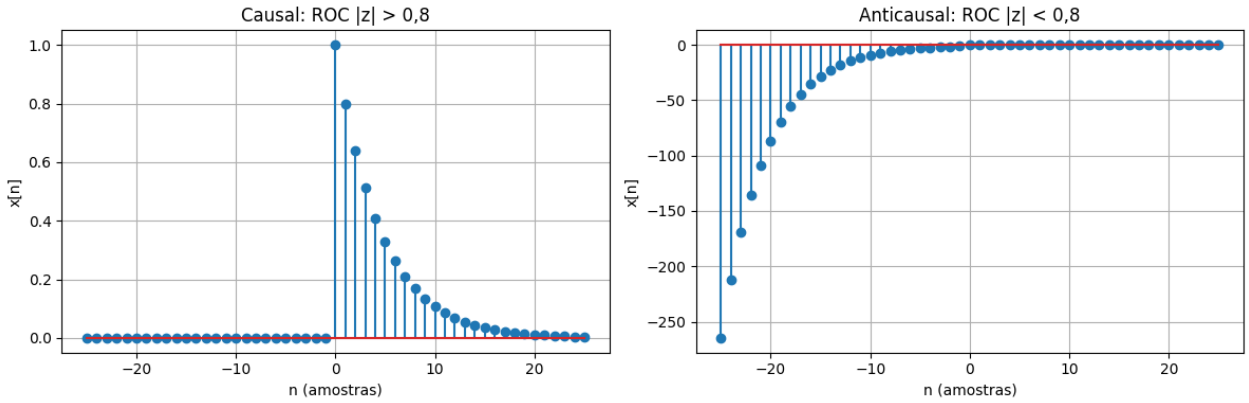
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
```

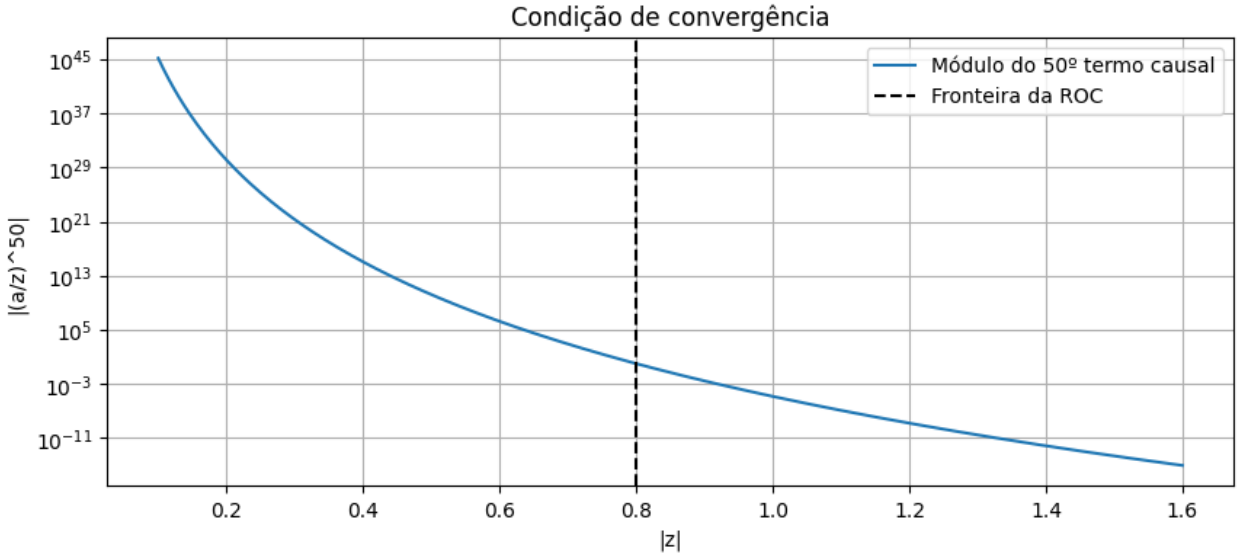
```
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

a=.8; n=np.arange( -25,26)
causal=np.where(n>=0,a**n,0); anticausal=np.where(n<0, -a**n,0)
fig,ax=plt.subplots(1,2,figsize=(12,4))
ax[0].stem(n,causal); ax[0].set(title='Causal: ROC |z| > 0,8',xlabel='n (amostras)',ylabel='x[n]')
ax[1].stem(n,anticausal); ax[1].set(title='Anticausal: ROC |z| < 0,8',xlabel='n (amostras)',ylabel='x[n]')
plt.tight_layout(); plt.show()
r=np.linspace(.1,1.6,400)
plt.plot(r,(a/r)**50,label='Módulo do 50º termo causal'); plt.axvline(a,color='k',ls=' - -',label='Fronteira
     da ROC'); plt.yscale('log'); plt.xlabel('|z|'); plt.ylabel('|(a/z)^50|'); plt.title('Condição de
    convergência'); plt.legend(); plt.show()
```





---

111 Resultados

As duas sequências têm a mesma forma racional, mas suportes temporais e comportamentos de estabilidade diferentes.

112 Interpretação e Discussão

Um gráfico com número finito de termos não prova convergência. A prova decorre da série geométrica: |a/z| <1 para o caso causal e |z/a| <1 para o anticausal.

113 Conclusões

As duas sequências têm a mesma forma racional, mas suportes temporais e comportamentos de estabilidade diferentes.

114 Função de Transferência Discreta

---

115 Resumo conceitual

Sob repouso inicial, a função de transferência discreta relaciona entrada e saída no domínio Z. O denominador descreve a realimentação e seus polos determinam os modos naturais.

---

116 Formulação matemática

$$y[n] = ay[n-1] + x[n] \Rightarrow H(z) = \frac{Y(z)}{X(z)} = \frac{1}{1 - az^{-1}}, \quad h[n] = a^n u[n]. \tag{58}$$

A ROC causal é $|z| > |a|$.

---

117 Exemplo

Simule o sistema do guia com a = 0,8 e compare polos 0,5, 0,9, 1,1 e -0,8, com entrada impulso e memória inicialmente nula.

---

118 Simulação

Simule o sistema do guia com a = 0,8 e compare polos 0,5, 0,9, 1,1 e -0,8, com entrada impulso e memória inicialmente nula.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

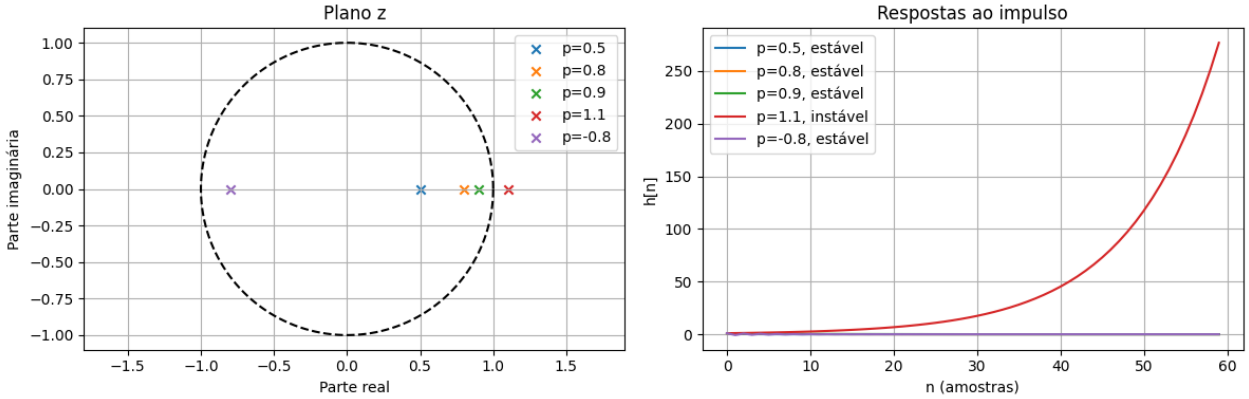
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
```

```
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

n=np.arange(60); fig,ax=plt.subplots(1,2,figsize=(12,4)); th=np.linspace(0,2*np.pi,500)
ax[0].plot(np.cos(th),np.sin(th),'k - -')
for p in [.5,.8,.9,1.1, -.8]:
    h=signal.lfilter([1],[1, -p],np.r_[1.,np.zeros(59)])
    assert np.allclose(h,p**n)
    ax[0].scatter(p,0,marker='x',label=f'p={p}')
    ax[1].plot(n,h,label=f'p={p}, '+('estável' if abs(p)<1 else 'instável'))
ax[0].axis('equal'); ax[0].set(xlabel='Parte real',ylabel='Parte imaginária',title='Plano z')
ax[1].set(xlabel='n (amostras)',ylabel='h[n]',title='Respostas ao impulso')
for a in ax: a.legend()
plt.tight_layout(); plt.show()
```



---

119 Resultados

Os polos positivos menores que 1 produzem decaimento; -0,8 produz alternância de sinais e 1,1 produz crescimento.

120 Interpretação e Discussão

Polos mais próximos do círculo unitário prolongam a memória. Um polo negativo estável não é instável por oscilar: o que importa é o módulo.

121 Conclusões

Os polos positivos menores que 1 produzem decaimento; -0,8 produz alternância de sinais e 1,1 produz crescimento.

122 Estabilidade no Plano z

---

123 Resumo conceitual

Um sistema LTI discreto é estável BIBO quando sua resposta ao impulso é absolutamente somável. Para sistemas causais racionais, isso corresponde a polos estritamente dentro do círculo unitário.

124 Formulação matemática

$$\sum_n |h[n]| < \infty. \quad h[n] = a^n u[n] \Rightarrow \sum_{n=0}^{\infty} |a|^n = \frac{1}{1-|a|}, \quad |a| < 1. \tag{59}$$

Polos sobre o círculo unitário não satisfazem estabilidade BIBO em geral.

---

125 Exemplo

Compare os polos do capítulo anterior, incluindo um caso instável, e confira a fórmula do impulso.

---

126 Simulação

Compare os polos do capítulo anterior, incluindo um caso instável, e confira a fórmula do impulso.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

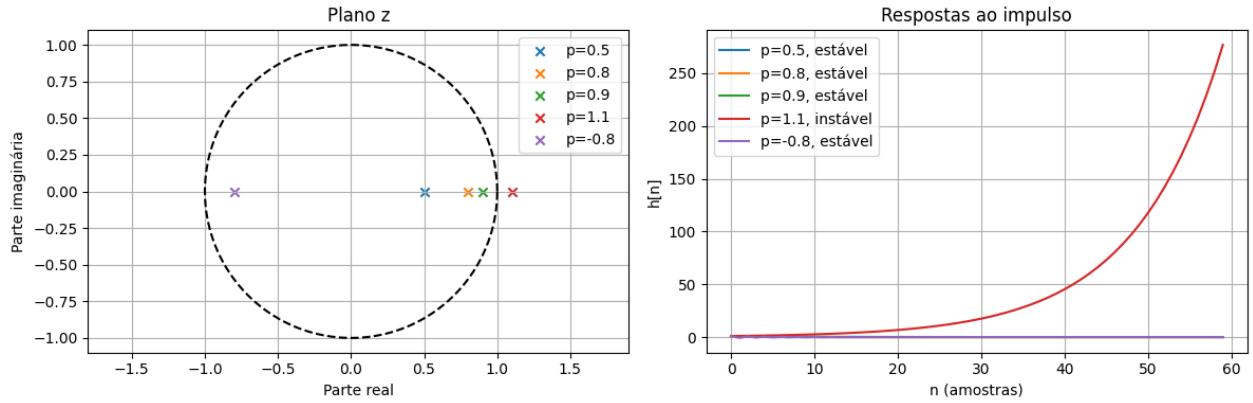
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
```

```
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

n=np.arange(60); fig,ax=plt.subplots(1,2,figsize=(12,4)); th=np.linspace(0,2*np.pi,500)
ax[0].plot(np.cos(th),np.sin(th),'k - -')
for p in [.5,.8,.9,1.1, -.8]:
    h=signal.lfilter([1],[1, -p],np.r_[1.,np.zeros(59)])
    assert np.allclose(h,p**n)
    ax[0].scatter(p,0,marker='x',label=f'p={p}')
    ax[1].plot(n,h,label=f'p={p}, '+('estável' if abs(p)<1 else 'instável'))
ax[0].axis('equal'); ax[0].set(xlabel='Parte real',ylabel='Parte imaginária',title='Plano z')
ax[1].set(xlabel='n (amostras)',ylabel='h[n]',title='Respostas ao impulso')
for a in ax: a.legend()
plt.tight_layout(); plt.show()
```



---

127 Resultados

O polo 0,9 decai mais lentamente que 0,5 e a resposta de 1,1 cresce sem limite.

128 Interpretação e Discussão

A análise de estabilidade não deve depender apenas de uma simulação curta. O critério analítico evita classificar um crescimento lento como estabilidade.

129 Conclusões

O polo 0,9 decai mais lentamente que 0,5 e a resposta de 1,1 cresce sem limite.

130 Relação entre Transformada Z e DTFT

---

131 Resumo conceitual

A DTFT corresponde à avaliação da transformada Z sobre o círculo unitário quando esse círculo pertence à ROC. A distância aos polos e zeros ajuda a interpretar a magnitude.

---

132 Formulação matemática

$$H(e^{j\omega}) = H(z)|_{z=e^{j\omega}}, \qquad H(z) = \frac{1 - z^{-1}}{1 - 0{,}8z^{-1}}. \tag{60}$$

O zero em z = 1 anula DC; a ROC causal $|z| > 0{,}8$ inclui o círculo unitário.

---

133 Exemplo

Analise b = [1, -1] e a = [1, -0,8] a fs = 1000 Hz. Compare a magnitude e o plano z.

---

134 Simulação

Analise b = [1, -1] e a = [1, -0,8] a fs = 1000 Hz. Compare a magnitude e o plano z.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

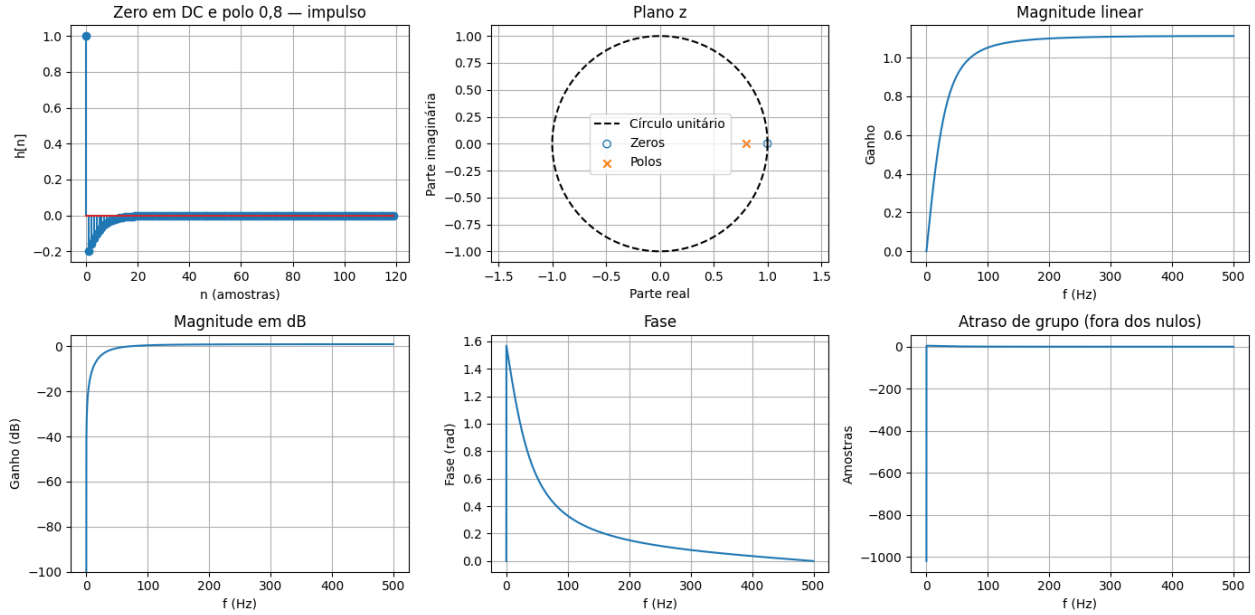
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
```

```
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

analisar([1, -1],[1, -.8],nome='Zero em DC e polo 0,8')
print('Ganho DC:',abs((1 -1)/(1 -.8)))
```




```
b = [ 1 -1]
a = [ 1.  -0.8]
Polos: [0.8] Estável causal: True
Ganho DC: 0.0
```

---

135 Resultados

A magnitude é zero em DC, e o plano z mostra o zero no ponto em que o círculo unitário cruza o eixo real positivo.

136 Interpretação e Discussão

Não basta substituir formalmente $z = e^{jw}$: é necessário verificar a ROC. Um sistema causal instável pode ter uma expressão racional finita sobre parte do círculo sem possuir DTFT da resposta ao impulso.

137 Conclusões

A magnitude é zero em DC, e o plano z mostra o zero no ponto em que o círculo unitário cruza o eixo real positivo.

138 Relação entre Laplace e Transformada Z

---

139 Resumo conceitual

O mapeamento exponencial associa modos contínuos amostrados aos modos discretos. O semiplano esquerdo é levado ao interior do círculo unitário.

---

140 Formulação matemática

$$z = e^{sT_s} = e^{\sigma T_s} e^{j\Omega T_s}. \tag{61}$$

Para $s = -2$ e $T_s = 0{,}1$ s, $z = e^{-0{,}2} \approx 0{,}81873$. O mapeamento de polos não define sozinho toda a função de transferência discretizada.

---

141 Exemplo

Compare s = -2, +1 e -1 ± j4 com Ts = 0,05; 0,1; 0,5 s.

---

142 Simulação

Compare s = -2, +1 e -1 ± j4 com Ts = 0,05; 0,1; 0,5 s.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

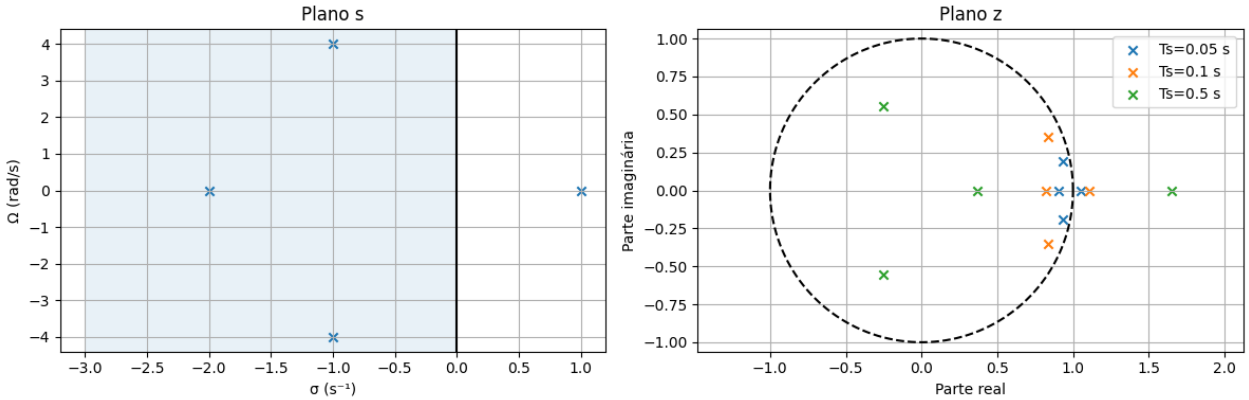
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
```

```
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

ps=np.array([ -2,1, -1+4j, -1 -4j]); fig,ax=plt.subplots(1,2,figsize=(12,4))
ax[0].axvspan( -3,0,alpha=.1); ax[0].scatter(ps.real,ps.imag,marker='x'); ax[0].axvline(0,color='k')
th=np.linspace(0,2*np.pi,500); ax[1].plot(np.cos(th),np.sin(th),'k - -')
for Ts in [.05,.1,.5]:
    pz=np.exp(ps*Ts); ax[1].scatter(pz.real,pz.imag,marker='x',label=f'Ts={Ts} s')
    print('Ts =',Ts,'s; polos z =',pz)
ax[0].set(xlabel=' \sigma (⁻s¹)',ylabel=' \Omega (rad/s)',title='Plano s'); ax[1].set(xlabel='Parte real',
     ylabel='Parte imaginária',title='Plano z'); ax[1].axis('equal'); ax[1].legend(); plt.tight_layout(); plt
     .show()
```

```
Ts = 0.05 s; polos z = [0.90483742+0.j         1.0512711 +0.j         0.93226817+0.18898011j
 0.93226817 -0.18898011j]
Ts = 0.1 s; polos z = [0.81873075+0.j         1.10517092+0.j         0.83341045+0.35236029j
 0.83341045 -0.35236029j]
Ts = 0.5 s; polos z = [ 0.36787944+0.j          1.64872127+0.j         -0.25240582+0.55151677j
 -0.25240582 -0.55151677j]
```



---

143 Resultados

Polos com parte real negativa permanecem dentro do círculo e o polo +1 fica fora dele. Aumentar Ts reduz o raio dos polos estáveis e aumenta o ângulo de modos oscilatórios.

144 Interpretação e Discussão

O ângulo discreto é periódico: frequências contínuas separadas por múltiplos de $2\pi/Ts$ coincidem. Métodos como retenção de ordem zero também definem numerador e ganho; não se deve discretizar um sistema apenas substituindo polos.

145 Conclusões

Polos com parte real negativa permanecem dentro do círculo e o polo +1 fica fora dele. Aumentar Ts reduz o raio dos polos estáveis e aumenta o ângulo de modos oscilatórios.

146 Mini Projeto Integrador — Análise Multidomínio de um Sensor

---

147 Resumo conceitual

O projeto investiga como a amostragem e a dinâmica de um sensor industrial alteram a identificação de duas componentes de vibração. O sinal é inteiramente sintético e permite conhecer a referência exata.

---

148 Formulação matemática

$$x(t) = \cos(2\pi 10t) + 0{,}5\cos(2\pi 70t + \pi/4), \quad H(s) = \frac{1}{0{,}02s + 1}. \tag{62}$$

Com retenção de ordem zero e $T_s = 1/500$ s, $p = e^{-50T_s}$ e $H_d(z) = (1-p)z^{-1}/(1-pz^{-1})$. A equação é $y[n] = py[n-1] + (1-p)x[n-1]$.

---

149 Exemplo

Observe 2 s a 500 Hz (Δf = 0,5 Hz), compare a aquisição a 100 Hz e estude um registro de 0,73 s para provocar vazamento. O modelo do sensor é um passa-baixas de primeira ordem com τ = 20 ms.

---

150 Simulação

Observe 2 s a 500 Hz (Δf = 0,5 Hz), compare a aquisição a 100 Hz e estude um registro de 0,73 s para provocar vazamento. O modelo do sensor é um passa-baixas de primeira ordem com τ = 20 ms.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
```

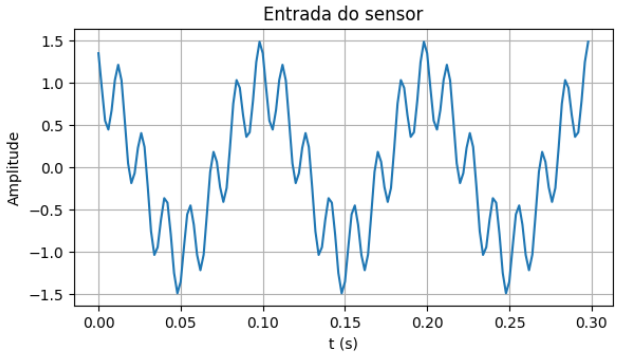
```
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

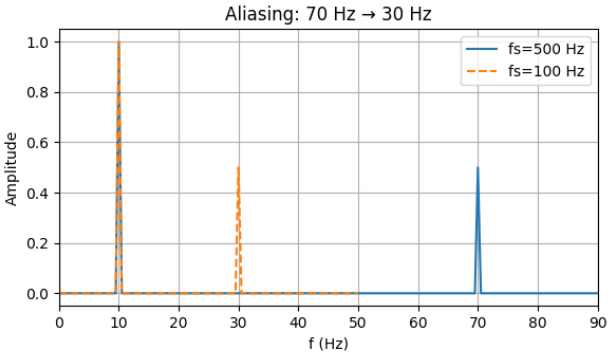
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

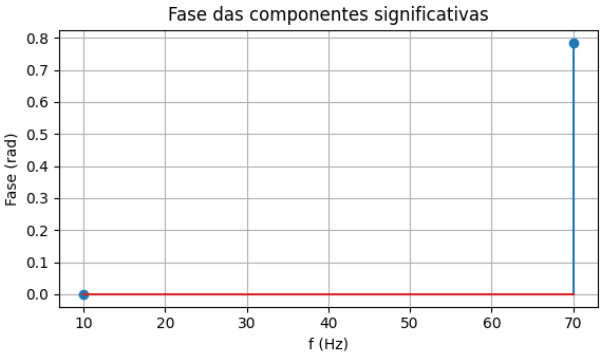
def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

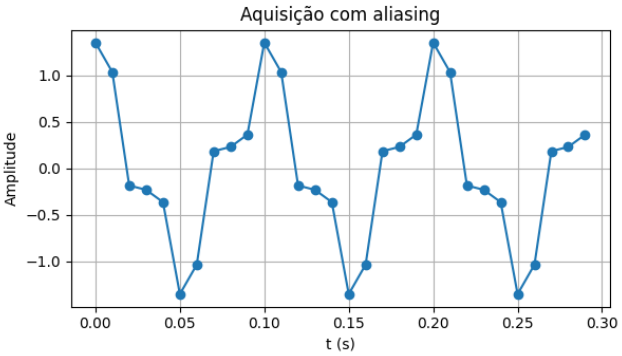
fs=500; t=np.arange(1000)/fs
x=np.cos(2*np.pi*10*t)+.5*np.cos(2*np.pi*70*t+np.pi/4)
fig,ax=plt.subplots(2,2,figsize=(12,7))
ax[0,0].plot(t[:150],x[:150]); ax[0,0].set(title='Entrada do sensor',xlabel='t (s)',ylabel='Amplitude')
f,A=espectro(x,fs); ax[0,1].plot(f,A,label='fs=500 Hz')
X=np.fft.rfft(x); mask=np.abs(X)>1e -7
ax[1,0].stem(f[mask],np.angle(X[mask])); ax[1,0].set(title='Fase das componentes significativas',xlabel='f (
    Hz)',ylabel='Fase (rad)')
ta=np.arange(200)/100; xa=np.cos(2*np.pi*10*ta)+.5*np.cos(2*np.pi*70*ta+np.pi/4)
fa,Aa=espectro(xa,100); ax[0,1].plot(fa,Aa,' - -',label='fs=100 Hz'); ax[0,1].set(xlim=(0,90),title='
    Aliasing: 70 Hz \rightarrow 30 Hz',xlabel='f (Hz)',ylabel='Amplitude'); ax[0,1].legend()
ax[1,1].plot(ta[:30],xa[:30],'o -'); ax[1,1].set(title='Aquisição com aliasing',xlabel='t (s)',ylabel='
    Amplitude')
plt.tight_layout(); plt.show()
N=365; xn=x[:N]
for nome,w in [('Retangular',np.ones(N)),('Hann',signal.windows.hann(N,sym=False)),('Hamming',signal.windows
    .hamming(N,sym=False))]:
    fw,Aw=espectro(xn,fs,w,4096); plt.plot(fw,20*np.log10(np.maximum(Aw,1e -8)),label=nome)
plt.xlim(0,100); plt.ylim( -80,5); plt.xlabel('f (Hz)'); plt.ylabel('Amplitude (dB)'); plt.title('Registro
    de 0,73 s: vazamento'); plt.legend(); plt.show()
tc=np.linspace(0,2,10001); xc=np.cos(2*np.pi*10*tc)+.5*np.cos(2*np.pi*70*tc+np.pi/4)
_,yc,_=signal.lsim(([1],[.02,1]),U=xc,T=tc)
b,a,dt=signal.cont2discrete(([1],[.02,1]),1/fs,method='zoh'); b=b.ravel()
p=np.exp( -50/fs)
assert np.allclose(np.roots(a),[p])
y=signal.lfilter(b,a,x)
plt.plot(tc[:2000],yc[:2000],label='Modelo contínuo, entrada contínua'); plt.plot(t[:200],y[:200],' - -',
    label='Modelo discreto, entrada retida')
plt.xlabel('t (s)'); plt.ylabel('Saída'); plt.title('Resposta do sensor'); plt.legend(); plt.show()
plt.scatter( -50,0,marker='x'); plt.axvline(0,color='k'); plt.axvspan( -60,0,alpha=.1); plt.xlabel(' \sigma
    (⁻s¹)'); plt.ylabel(' \Omega (rad/s)'); plt.title('Polo contínuo do sensor: -50'); plt.show()
analisar(b,a,fs,'Sensor discretizado por ZOH')
print('fs=500 Hz > 2 \times70 Hz; \Deltaf=0,5 Hz; polo z =',p)
print('Picos identificados:',f[np.argsort(A)[ -2:]],'Hz')
for fi in [10,70]:
```

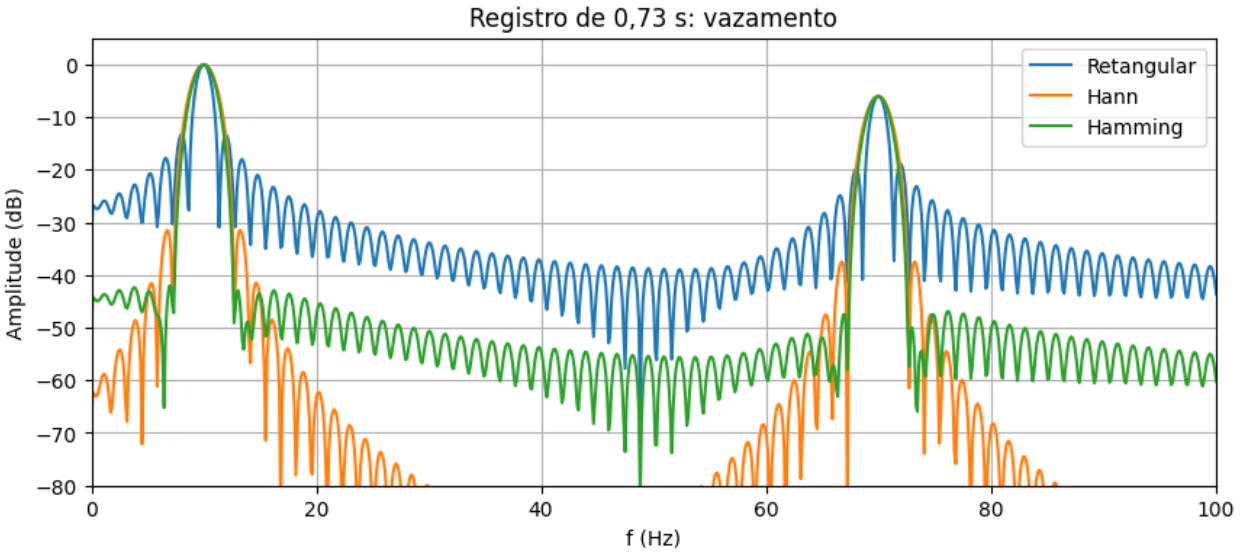
```
ganho=1/np.sqrt(1+(2*np.pi*fi*.02)**2)
print(fi,'Hz: ganho contínuo =',ganho)
```

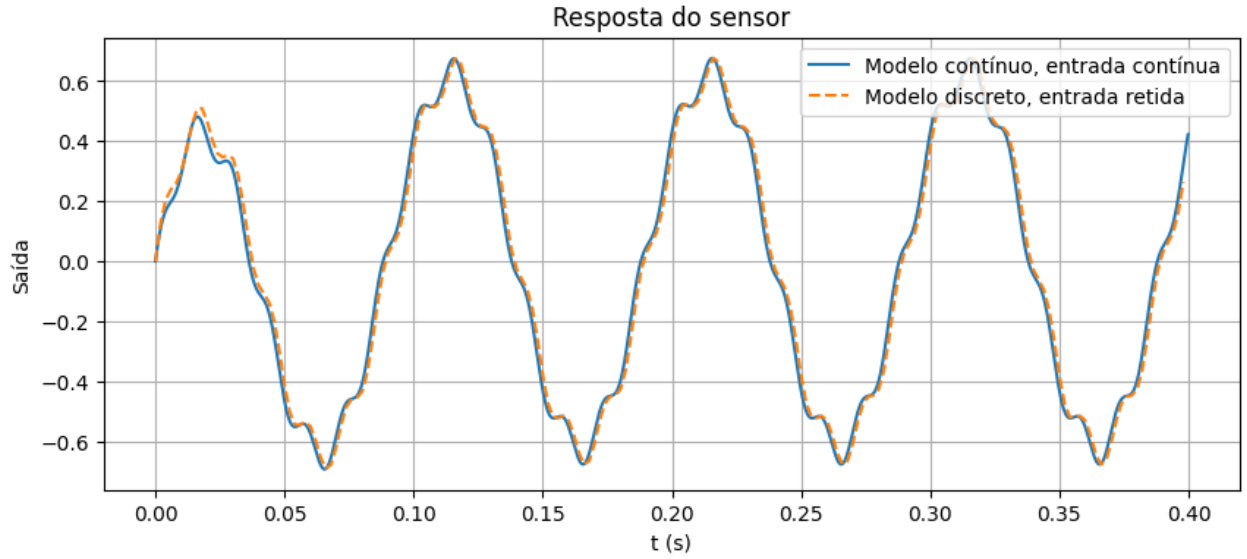


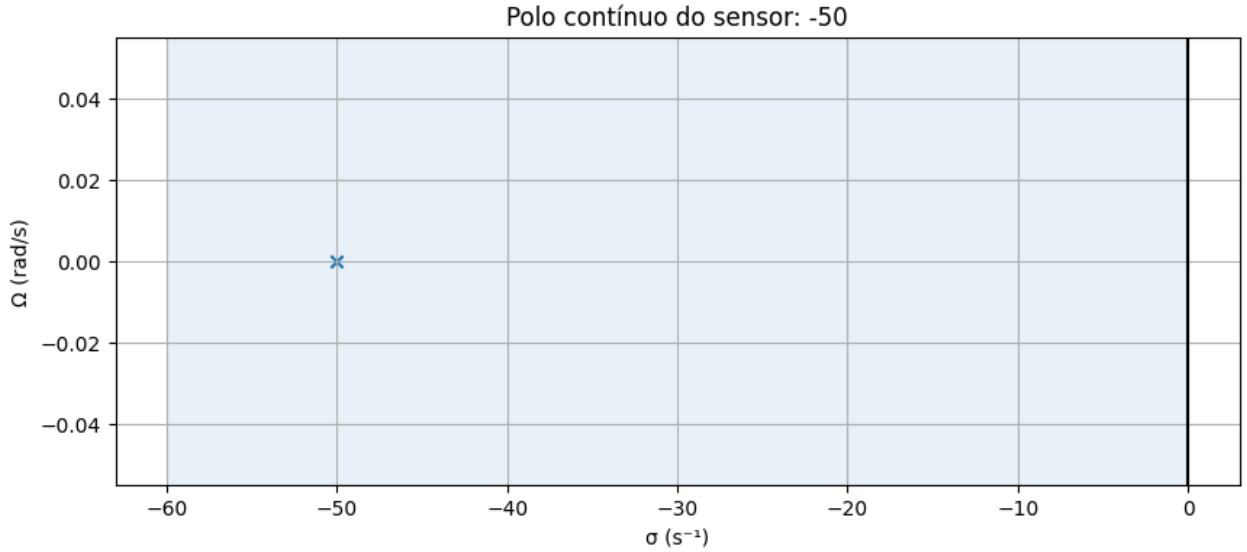


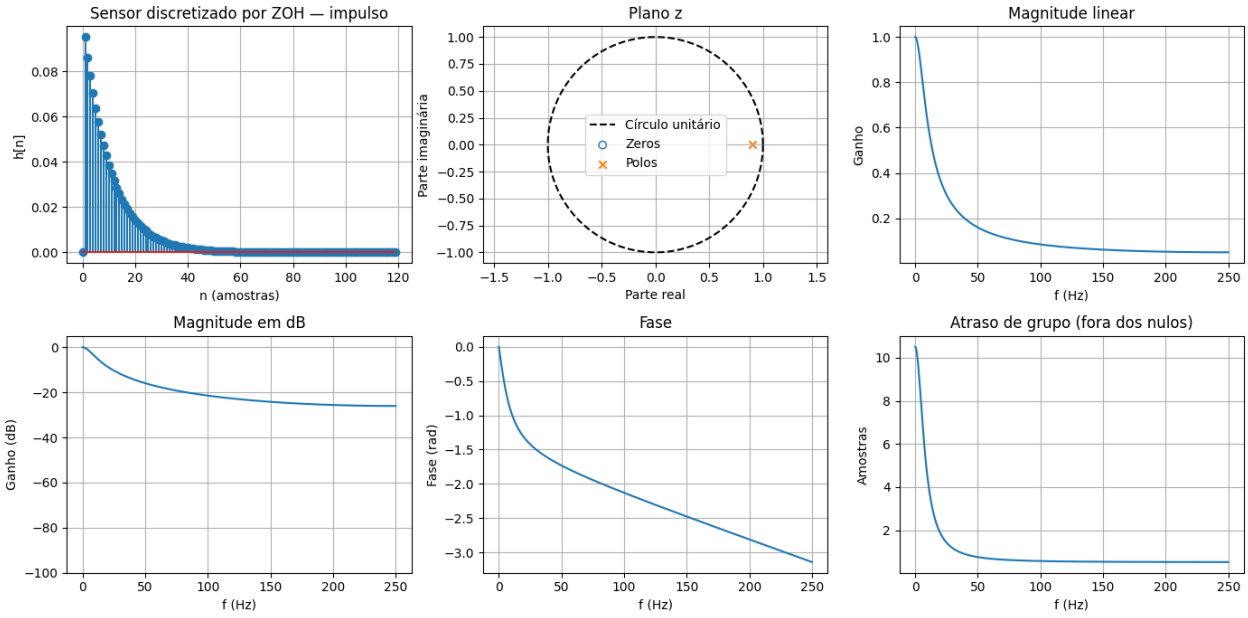













```
b = [0.         0.09516258]
a = [ 1.         -0.90483742]
Polos: [0.90483742] Estável causal: True
fs=500 Hz > 2 \times70 Hz; \Deltaf=0,5 Hz; polo z = 0.9048374180359595
Picos identificados: [70. 10.] Hz
```

```
10 Hz: ganho contínuo = 0.6226769922994999
70 Hz: ganho contínuo = 0.11295455441066325
```

---

151 Resultados

O espectro identifica 10 e 70 Hz com amplitudes 1 e 0,5 e fases 0 e $\pi/4$. A aquisição a 100 Hz desloca 70 para 30 Hz. O polo do sensor é -50 s^{-1}, mapeado em 0,90484, e os ganhos contínuos são aproximadamente 0,623 e 0,113 nas duas frequências.

152 Interpretação e Discussão

O tempo mostra a composição e os transientes, enquanto a frequência separa as componentes. A dinâmica do sensor reduz principalmente 70 Hz, mesmo com aquisição adequada. A discretização ZOH é exata para entradas constantes entre amostras; por isso sua saída pode diferir da resposta à entrada contínua senoidal. Hann e Hamming reduzem os lóbulos laterais do registro não coerente, à custa de largura do lóbulo principal. O modelo é estável nos dois planos e a observação de polos não substitui a análise de ganho e banda útil.

153 Conclusões

O espectro identifica 10 e 70 Hz com amplitudes 1 e 0,5 e fases 0 e $\pi/4$. A aquisição a 100 Hz desloca 70 para 30 Hz. O polo do sensor é -50 s^{-1}, mapeado em 0,90484, e os ganhos contínuos são aproximadamente 0,623 e 0,113 nas duas frequências.

153.1 Conclusões da Parte 2

A análise espectral distingue componentes sobrepostas no tempo, mas depende da duração do registro, da janela e da taxa de amostragem. Laplace e Z explicam a dinâmica e a estabilidade, sempre com as hipóteses de causalidade e convergência. O projeto do sensor evidencia que aquisição correta não elimina as limitações de banda do sistema.

154 Referências bibliográficas

- SILVA, Moacy Pereira da. **Estudo Dirigido de Processamento Digital de Sinais — Parte 2**. IFPB, Campus Campina Grande, 2026.2. Guia fornecido em `docs/guia_parte_2.pdf`; fonte do roteiro e das investigações.
- OPPENHEIM, A. V.; SCHAFER, R. W. **Discrete-Time Signal Processing**. 3. ed. Pearson, 2010. Fundamentação de sinais, transformadas e filtros.
- PROAKIS, J. G.; MANOLAKIS, D. G. **Digital Signal Processing: Principles, Algorithms, and Applications**. 4. ed. Pearson, 2007.
- OPPENHEIM, A. V.; WILLSKY, A. S.; NAWAB, S. H. **Signals and Systems**. 2. ed. Prentice Hall, 1997. Sistemas contínuos e discretos, convergência e estabilidade.
- NUMPY. Discrete Fourier Transform. Convenções de DFT e frequências dos coeficientes. Acesso em: 8 nov. 2026.
- SCIPY. Signal processing. Rotinas de análise, discretização e projeto utilizadas nos notebooks. Acesso em: 8 nov. 2026.

Os guias definem o conteúdo; os livros são referências de aprofundamento. Os dados, exemplos numéricos e códigos deste estudo foram construídos para as simulações, sem aquisição experimental.